Berliner Hochschule für Technik

Fachbereich VI - Informatik und Medien

Bachelor's Thesis

Cost-Sensitive Fraud Detection in Financial Transactions:

A Comparative Analysis of XGBoost and Neural Networks to Reduce False Negatives

Mohamed Aziz Hassine
Matrikelnummer: 100894
Computer Engineering (B.Eng.)
Born on 04 June 2001
in Ksibet Mediouni, Tunisia

Supervisor: M.Sc. Aljoscha Marcel Everding
Reviewer: Prof. Dr. Thomas Off

Submission date:
07 September 2026

Abstract

Credit card fraud causes substantial financial losses for financial institutions, and false negatives – fraudulent transactions that go undetected – are the most costly classification errors, as they directly result in financial losses. Standard machine learning models are typically optimized for aggregate metrics that weight all errors equally, which does not reflect the asymmetric cost structure of fraud detection, where missing a fraudulent transaction is substantially more costly than incorrectly flagging a legitimate one. This thesis investigates cost-sensitive learning strategies for fraud detection, comparing standard cost-insensitive baseline models with cost-sensitive configurations, with the goal of reducing false negatives without excessively increasing the number of false positives. Two machine learning model classes are compared: XGBoost, a tree-based gradient boosting method, and neural networks. Both are trained under a standard cost-insensitive configuration and under cost-sensitive configurations. The comparison is performed using two publicly available datasets with different characteristics: the IEEE-CIS Fraud Detection dataset [20] and the ULB Credit Card Fraud dataset [13]. Without cost sensitivity, both models achieve high precision but limited recall, and XGBoost consistently outperforms the neural network, clearly so on the noisier IEEE-CIS data and by a smaller margin on the ULB data. Introducing cost sensitivity shifts the balance toward catching fraud: a cost optimal decision threshold reduces false negatives below the baseline on ULB while keeping precision high, and reduces them substantially on IEEE-CIS, though there only by causing more false alarms. The results show that cost-sensitive learning can reduce the most costly errors in fraud detection, but whether this comes at little or high cost depends on the dataset and the chosen decision rule.

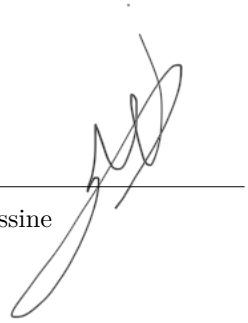
Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Stellen der Arbeit, die wörtlich oder sinngemäß aus veröffentlichten oder unveröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Declaration of Authorship

I hereby declare that I have written this thesis independently and without the use of any aids other than those specified. All passages taken literally or in substance from published or unpublished sources are marked as such. This work has not previously been submitted, in the same or similar form, to any other examination authority, nor has it been published.

Berlin, den 07 September 2026



Mohamed Aziz Hassine

Contents

List of Figures v

List of Tables vi

List of Listings vii

1 Introduction 1
- 1.1 Motivation and Problem Statement . . . 1
- 1.2 Relevance of Fraud Detection in Financial Transactions . . . 1
- 1.3 Research Objectives . . . 2
- 1.4 Hypotheses . . . 3
- 1.5 Scope and Limitations . . . 4
- 1.6 Structure of the Thesis . . . 4

2 Theoretical Background 5
- 2.1 Fraud Detection in Financial Systems . . . 5
- 2.2 Machine Learning Approaches to Fraud Detection . . . 5
- 2.3 Characteristics of Financial Transaction Fraud Data . . . 6
- 2.4 The Class Imbalance Problem . . . 7

3 Cost-Sensitive Machine Learning 12
- 3.1 From Accuracy to Expected Loss . . . 12
- 3.2 The Cost Matrix . . . 13
- 3.3 The Cost-Sensitive Optimal Threshold . . . 14
- 3.4 Implementation Strategies . . . 15
- 3.5 Algorithm-Specific Considerations . . . 15
- 3.6 Summary . . . 16

4 Machine Learning Models 17
- 4.1 XGBoost . . . 17
- 4.2 Neural Networks for Tabular Data . . . 18
- 4.3 Hyperparameter Tuning and Training Protocol . . . 20
- 4.4 Comparison: Strengths and Weaknesses for Tabular Fraud Detection . . . 22

5 Data and Methodology 24
- 5.1 Datasets . . . 24
- 5.2 Exploratory Data Analysis . . . 25
- 5.3 Missing Data Methodology . . . 27
- 5.4 Preprocessing Pipeline . . . 30
- 5.5 Baseline Modelling . . . 33

6 Cost-Sensitive Results 36
- 6.1 Experimental Protocol and Cost-Based Metrics . . . 36

6.2 Experimental Results 38
6.3 Comparison: XGBoost vs. Neural Network 41
6.4 Comparison: IEEE-CIS vs. ULB 41

7 Discussion **42**
7.1 Summary of Findings 42
7.2 Interpretation 43
7.3 Limitations 44

8 Conclusion **45**

Software and Tools Used **49**

List of Figures

1.1 Global losses due to payment-card fraud from 2014 to 2023, including forecasts through 2028, split between the United States and the rest of the world [19]. 2

2.1 Class distribution comparison on a logarithmic scale (raw datasets, before deduplication), highlighting the severe class imbalance in the IEEE-CIS dataset (left) and the ULB dataset (right). 8

4.1 Baseline neural network architecture for IEEE-CIS fraud detection. Representative neurons are shown; dots indicate omitted neurons. 19

5.1 Structure of the datasets. Left: identity-record availability in IEEE-CIS. Right: feature-space dimensionality of the ULB, IEEE-CIS transaction, and IEEE-CIS identity tables. 24

5.2 Fraud rate by transaction characteristic in IEEE-CIS (merged transaction and identity data): product category, card type, device type, and the presence of an identity record. Bars show the univariate fraud rate within each category against the dataset baseline. 26

5.3 Fraud rate by purchaser email domain in IEEE-CIS, restricted to domains with at least 50 transactions. `protonmail.com` is the single strongest interpretable predictor identified, far above mainstream providers such as `gmail.com` and `yahoo.fr`. 27

5.4 Leakage-aware preprocessing and evaluation pipeline. Transformations are fitted exclusively on the training split, thresholds are tuned on validation data, and final metrics are evaluated on the test partition. 30

List of Tables

3.1 General cost matrix for binary fraud classification. 13

4.1 Search space for the XGBoost randomized search (30 iterations, three-fold stratified cross-validation, scored on PR-AUC). 21
4.2 Best XGBoost configurations selected by randomized search, per dataset, with corresponding cross-validated PR-AUC. 21
4.3 Comparison of XGBoost and neural network properties in tabular fraud detection. 23

5.1 Strongest informative-missingness signals among the 174 high-missingness columns in `train_transaction.csv`. Fraud prevalence is contrasted between missing and present observations against baseline prevalence. 28
5.2 Distribution of relative missingness gap across the 174 high-missingness columns in `train_transaction.csv`. 29
5.3 Baseline classification performance evaluated at $t = 0.5$ without cost-sensitive adjustments on the held-out test partitions. 34

6.1 First attempt: baseline versus the highest-savings configuration under the amount-dependent rule ($p^*(x) = C_a/x$, $C_a = 2.50\,\text{EUR}$). FN = missed frauds, FP = false alarms. 38
6.2 Refined result: baseline versus the best false-negative-reducing configuration using the global threshold t^* (with the C_a at which it is obtained). XGB*=XGBoost, NN*=Neural Network 39
6.3 Best achievable savings as a function of the administrative cost C_a (EUR). . 40

List of Listings

5.1 Gap-based informative-missingness test: for each high-missingness column, the fraud rate among missing versus present values is compared against the baseline fraud rate. 29
5.2 Evaluation of feature cardinality and identification of categorical columns. . 31
5.3 Categorical imputation and one-hot encoding prior to partitioning. 32
5.4 Leakage-aware pipeline: the split precedes all fitting; imputation medians and the scaler are learned on the training partition only and applied unchanged to validation and test. 32

6.1 Selection of the global cost-optimal threshold t^* on the validation partition. 37

AUC-PR	Area Under the Precision - Recall Curve
BCE	Binary Cross-Entropy
CV	Cross-Validation
FN	False Negative
FP	False Positive
IEEE-CIS	IEEE Computational Intelligence Society (Fraud Detection dataset)
NN	Neural Network
PCA	Principal Component Analysis
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
TN	True Negative
TP	True Positive
ULB	Université Libre de Bruxelles (Credit Card Fraud dataset)
XGBoost	Extreme Gradient Boosting

1 Introduction

1.1 Motivation and Problem Statement

Fraud detection in financial transactions is a critical task for banks and other financial institutions, given the significant financial losses, reputational damage, and operational disruptions that fraudulent activity can cause. Beyond direct economic impact, regulatory requirements increasingly mandate effective fraud prevention measures. While false negatives (fraudulent transactions that go undetected) pose a more costly financial threat by directly resulting in monetary losses [16], false positives present an equally critical challenge. Falsely declining a legitimate transaction causes customer dissatisfaction and can harm the institution's reputation [1].

Machine learning models are widely used for automated fraud detection, but standard classifiers are typically trained and evaluated with the implicit assumption that all classification errors are equally costly. This assumption does not reflect the reality of fraud detection, where the cost of missing a fraudulent transaction is generally far greater than the cost of incorrectly flagging a legitimate one. A classifier that treats both errors equally therefore optimizes the wrong objective: it minimizes the number of misclassifications rather than their real monetary cost, even though the loss from a single missed fraud can range from a few to several thousand euros [7].

This thesis studies *cost-sensitive learning* as a way to close the gap between what a classifier optimizes and what actually matters in fraud detection. Instead of treating every misclassification alike, cost-sensitive learning assigns an explicit cost to each type of error and trains or adjusts the model to minimize the total expected cost rather than the raw error count [10]. The thesis asks two questions. First, how far can cost-sensitive methods reduce false negatives in fraud detection without causing too many false positives? Second, does their effectiveness differ between two very different model types (XGBoost and neural networks) and between datasets with different characteristics?

1.2 Relevance of Fraud Detection in Financial Transactions

Global payment-card fraud losses reached approximately 33.8 billion U.S. dollars in 2023 and are projected to exceed 43 billion by 2028 [19]. For financial institutions, missed fraud means direct monetary loss, alongside operational overhead and reputational damage. Au-

tomated fraud detection is therefore a necessary layer of financial infrastructure, meaning that even incremental improvements in detection performance directly reduce financial exposure.

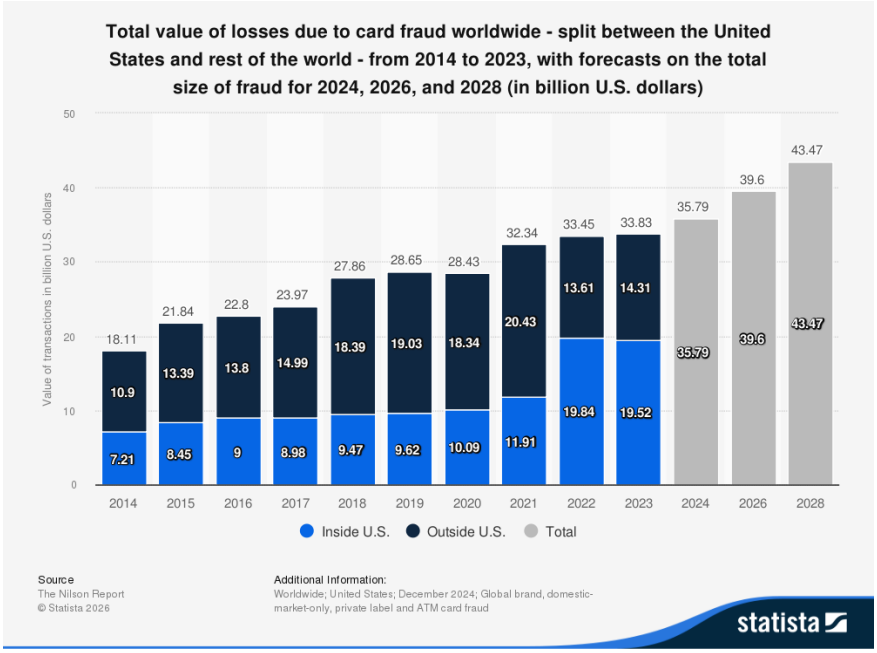


Figure 1.1: Global losses due to payment-card fraud from 2014 to 2023, including forecasts through 2028, split between the United States and the rest of the world [19].

At the same time, fraud detection poses distinct technical challenges for machine learning. Because fraudulent transactions are inherently rare, standard training objectives suffer from severe class imbalance (Section 2.4). The available data is also difficult in ways that differ from one dataset to the next: the ULB dataset is fully anonymized through PCA, so its features carry no human-interpretable meaning, while the IEEE-CIS dataset contains extensive missing values that cannot simply be dropped without losing information (Section 5.3). To account for these differences, this thesis evaluates cost-sensitive learning on two datasets with contrasting class imbalance, feature transparency, and missingness, testing how well the strategies hold up under different data conditions.

1.3 Research Objectives

The objective of this thesis is the systematic investigation of cost-sensitive learning strategies for fraud detection in financial transactions, with particular emphasis on reducing false negatives. Since undetected fraudulent transactions represent the most costly outcome for financial institutions, the emphasis of this work lies not on overall classification accuracy, but on cost-oriented model evaluation, formalized through the expected-loss framework introduced in Chapter 3.

Two model classes are examined: the gradient boosting method XGBoost and artificial Neural Networks. Both are trained in a standard, cost-insensitive configuration to establish baseline performance, and subsequently under multiple cost-sensitive strategies. The central research questions this thesis addresses are:

1. To what extent do cost-sensitive learning strategies reduce false negatives relative to cost-insensitive baseline models, and at what cost to false positive rates?
2. How does the effectiveness of cost-sensitive strategies differ between XGBoost and Neural Networks?
3. Do these findings generalize across datasets with different class imbalance severity and data quality characteristics?

By comparing standard and cost-sensitive training regimes across both model classes and both datasets, this thesis aims to provide insight into how cost-sensitive methods affect model behavior in practice, and to offer practical guidance on their application in real-world fraud detection systems.

1.4 Hypotheses

Before the experiments were carried out, three expectations guided the design of this study. Stating them explicitly here allows them to be tested against the results reported in Chapters 5 and 6 and revisited in the discussion.

H1 – Cost-sensitivity reduces false negatives, at a cost. Cost-sensitive learning was expected to reduce the number of false negatives relative to the cost-insensitive baselines, in line with the expected-loss principle of Elkan [10] and the amount-based cost formulation of Bahnsen et al. [7]. This reduction was, however, expected to come at a price: catching more fraud by lowering the decision threshold should also raise the number of false positives, so an improvement in recall was anticipated to trade off against precision rather than to be obtained for free.

H2 – XGBoost outperforms the neural network on this tabular task. XGBoost was expected to be the stronger of the two models, both at the cost-insensitive baseline and under cost-sensitive adaptation. This expectation follows the substantial body of evidence that gradient-boosted trees remain highly competitive with, and frequently superior to, neural networks on structured tabular data [11], particularly on data with missing values and mixed feature types such as those studied here.

H3 – The larger dataset with more fraud cases yields the better results. The IEEE-CIS dataset was expected to yield the stronger and more reliable classification performance,

since it is considerably larger than ULB (590,540 versus 284,807 transactions) and contains far more fraudulent examples to learn from (20,663 versus 492). A larger number of positive cases was expected to give the models a more complete picture of fraudulent behaviour and thus a clear advantage on IEEE-CIS.

1.5 Scope and Limitations

This work is limited to transaction-based fraud detection using tabular datasets, with a primary focus on payment-card transactions, encompassing both credit and debit cards. Other fraud types, such as identity theft, network-based fraud structures, or money laundering, are not considered. Only supervised learning methods are examined; unsupervised anomaly detection, graph-based models, and explainable AI approaches fall outside the scope of this thesis. The focus is on model development and evaluation under cost-sensitive conditions; aspects such as system integration, large-scale real-time deployment, or long-term adaptation to evolving fraud patterns (concept drift) are not addressed. The aim throughout is a controlled experimental analysis under comparable training conditions, rather than a production-ready fraud detection system.

1.6 Structure of the Thesis

The remainder of this thesis is organized as follows. Chapter 2 introduces the theoretical background of fraud detection, class imbalance, and evaluation metrics for imbalanced classification. Chapter 3 develops the formal cost-sensitive learning framework applied throughout this thesis, including the expected-loss formulation and threshold-moving and rebalancing strategies. Chapter 4 introduces XGBoost and Neural Networks as the two model classes under comparison, discussing their theoretical properties and suitability for tabular fraud data. Chapter 5 describes the datasets used, the exploratory data analysis conducted, the preprocessing methodology developed for handling missing data, and the results of cost-insensitive baseline models. Chapter 6 presents the cost-sensitive experiments and results, including the comparison between XGBoost and Neural Networks under multiple cost scenarios. Chapter 7 interprets these results, discusses their practical implications, and addresses the limitations of the study. Chapter 8 summarizes the thesis's findings and outlines directions for future research.

2 Theoretical Background

This chapter establishes the concepts the rest of the thesis builds on. It frames fraud detection as a machine learning problem, surveys the approaches applied to it, and examines what makes financial transaction data hard to classify, in particular severe class imbalance and the evaluation metrics suited to it. Together these form the basis for the cost-sensitive framework of Chapter 3 and the methodological choices of Chapter 5.

2.1 Fraud Detection in Financial Systems

Fraud is broadly defined as the crime of cheating someone to obtain money or goods illegally. From a machine-learning perspective, the detection of fraudulent transactions is treated as a binary classification problem, in which a model uses the features of a transaction to predict whether it is fraudulent or legitimate [3]. Financial institutions increasingly rely on automated, machine-learning-based fraud detection as part of their detection systems, alongside expert rules and human investigators. One reason is the large volume of digital transactions processed every day, which cannot be reviewed manually. Another is that fraud patterns change over time, so detection systems must be able to recognize new and previously unseen fraudulent behaviour [8]. When fraud goes undetected, the financial institution or cardholder can suffer a direct financial loss. On the other hand, flagging too many legitimate transactions as fraudulent can also cause problems. These false positives require additional manual verification and can result in the rejection or delay of legitimate transactions, which can negatively impact the customer experience [1]. Therefore, false negatives and false positives carry fundamentally asymmetric operational consequences. Accordingly, the primary focus of this thesis is to minimize costly false negatives to prevent fraud losses, while simultaneously constraining the volume of false positives to preserve operational efficiency and maintain high precision.

2.2 Machine Learning Approaches to Fraud Detection

Early fraud detection systems were mainly based on manually defined rules and thresholds created by domain experts. Although these systems are relatively easy to understand, they have some important limitations. The rules need to be updated manually and can become ineffective when fraudsters adapt their behaviour or learn how the detection system works.

Over time, research in this area has therefore moved from static rule-based systems towards data-driven machine learning methods. These methods can identify patterns in large numbers of transactions and can be updated as new data becomes available [1]. However, the use of machine learning does not remove the main challenges of fraud detection. In particular, the strong class imbalance and the changing nature of fraudulent behaviour remain important issues. A wide range of supervised learning methods has been used for credit card fraud detection. These include logistic regression, support vector machines, tree-based ensemble methods, and more recently, deep neural networks. Alarfaj et al. [2] compare several of these approaches, including gradient boosting and convolutional neural networks, using the same European credit card dataset considered in this thesis. Their results show that deep learning models can achieve higher detection performance, but they also require more extensive preprocessing and tuning. In addition, the data had to be rebalanced and additional measures were needed to reduce the number of missed fraudulent transactions [2]. Their study shows the benefits of comparing tree-based and neural network models directly, while also highlighting the additional effort required when using neural networks for tabular data. Another important aspect of fraud detection is the difference between offline evaluation and real-world applications. Dal Pozzolo et al. point out that practical fraud detection systems have to deal with concept drift as well as *verification latency*, meaning the time between a transaction and the point at which its actual label becomes available. They therefore propose learning methods that take these conditions into account [8]. This thesis focuses on controlled offline experiments (Section 1.5) and does not address these issues directly. Nevertheless, this previous work highlights the importance of developing models that can generalise to new data instead of simply fitting a fixed set of historical transactions. Cost-sensitive learning has also been studied specifically in the context of fraud detection. Bahnsen et al. and Sahin et al. use a monetary cost perspective instead of relying mainly on accuracy or AUC, with differences in how the cost of missed fraud is determined [7], [17]. Other studies have also considered the calibration of model probabilities before applying cost-based decisions [9]. These approaches provide the basis for the cost-sensitive framework developed in Chapter 3. However, there is less research that directly compares XGBoost and neural networks using explicit cost-sensitive operating points. This thesis therefore focuses on such a comparison using two datasets with substantially different levels of class imbalance and differences in data quality.

2.3 Characteristics of Financial Transaction Fraud Data

Fraud datasets at the transaction level, covering both credit and debit transactions, have several common characteristics that distinguish them from standard supervised learning datasets:

- **Severe class imbalance.** Fraudulent transactions are, by nature, rare relative to

legitimate ones, often constituting well under 5% of all transactions. Both datasets used in this thesis exhibit this property, though to markedly different degrees (Section 5.1).

- **Anonymization and confidentiality constraints.** Financial institutions cannot disclose raw transaction features without violating customer privacy, leading to widespread use of anonymized or PCA* transformed features in publicly available fraud datasets, as is the case for the majority of the ULB* dataset's features.
- **Mixed interpretability.** Some publicly released datasets, such as IEEE-CIS*, retain a subset of interpretable features (e.g., product category, device type, email domain) alongside anonymized ones, allowing partial insight into which transaction characteristics are associated with fraud, as explored in the exploratory analysis of Section 5.2.
- **Structured missingness.** Real-world transaction data frequently contains substantial missing data, arising from optional fields, incomplete device or identity metadata, or systematic differences in how data is collected across channels. Section 5.3 addresses this characteristic in detail for the IEEE-CIS* dataset.

2.4 The Class Imbalance Problem

Class imbalance arises when one class in a classification problem occurs far less often than the other. In fraud detection it is inherent to the problem rather than a by-product of data collection, since fraudulent transactions are genuinely rare. A model trained without any correction (one that simply minimises overall error or cross-entropy loss) will drift toward the majority class, because labelling the many legitimate transactions correctly lowers the total loss far more than labelling the few fraudulent ones correctly. In the limiting case, a model that predicts "legitimate" for every transaction still reaches 99.83% accuracy on the ULB dataset (Section 5.1) while catching no fraud at all. Figure 2.1 shows how severe this imbalance is in both datasets, and it is the reason accuracy on its own is an inadequate objective for the task [12].

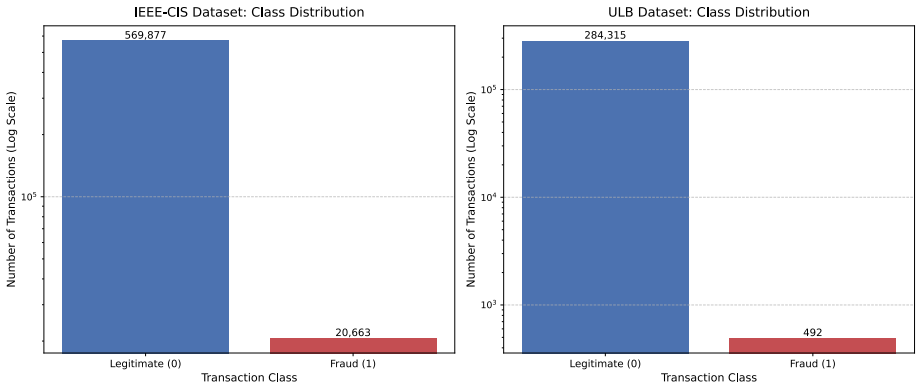


Figure 2.1: Class distribution comparison on a logarithmic scale (raw datasets, before deduplication), highlighting the severe class imbalance in the IEEE-CIS dataset (left) and the ULB dataset (right).

Approaches to handling class imbalance can generally be divided into two groups, depending on where the imbalance is addressed in the learning process [1], [8]: *data-level* methods and *algorithm-level* methods. Data-level methods change the training data before the model is fitted. This can be done by reducing the number of majority-class samples, increasing the number of minority-class samples, or creating synthetic minority examples, such as with SMOTE [8]. These methods can help create a more balanced training set, but they also change the original distribution of the data. Depending on how the samples are selected or generated, this can introduce bias or create artificial patterns that are not present in the original data.

Algorithm-level methods take a different approach. Instead of changing the training data, they keep the original class distribution and modify how the model learns or makes predictions [1], [8]. This can include giving more weight to minority-class errors during training or changing the decision threshold used to classify transactions.

This thesis focuses on algorithm-level approaches and therefore keeps the original, imbalanced transaction distribution.

2.4.1 Precision and Recall

Precision and recall provide two different views of a fraud detection model's performance. Precision measures the proportion of transactions classified as fraudulent that are actually fraudulent, while recall measures the proportion of fraudulent transactions that are correctly identified:

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN} \tag{2.1}$$

where TP, FP, and FN denote true positives, false positives, and false negatives, respec-

tively. Recall is particularly important in this thesis because failing to identify a fraudulent transaction is a central concern. It indicates how much of the actual fraud is detected by the model. Precision provides a complementary measure by showing how many of the transactions flagged as fraudulent are actually fraud. A low precision therefore means that the system produces a large number of false alarms.

Neither metric is sufficient on its own. A model could achieve high recall by flagging a very large number of transactions as fraudulent, but this would likely result in many false positives and therefore low precision. Conversely, a model could achieve high precision by being very conservative, but miss a large proportion of the actual fraud. The appropriate balance between these outcomes depends on the costs associated with each type of error.

2.4.2 F1 Score

The F1 score combines precision and recall into a single measure using their harmonic mean:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2.2}$$

A high F1 score requires both precision and recall to be reasonably high. This makes it useful for comparing models when both types of performance are considered important. However, F1 gives equal importance to precision and recall and does not take the financial consequences of different errors into account. This is an important limitation in the context of this thesis, where false positives and false negatives may have substantially different costs. The cost-sensitive framework introduced in Chapter 3 addresses this by explicitly incorporating these costs into the decision process.

2.4.3 Precision-Recall AUC

The area under the precision-recall curve (AUC-PR) summarises the relationship between precision and recall over a range of decision thresholds. Instead of evaluating a model at a single threshold, it considers how its performance changes as the threshold is varied.

This is particularly useful for the datasets considered in this thesis because fraudulent transactions form a small minority of all transactions. AUC-PR therefore provides a threshold-independent measure that focuses on the model's ability to identify the minority class while accounting for the false positives generated in doing so [12].

AUC-PR is used here primarily to compare the general ranking performance of different models. The threshold used to make the final fraud or legitimate decision is considered separately, since the threshold determines the actual trade-off between false positives and false negatives.

2.4.4 Why Precision–Recall over ROC under Severe Imbalance

The Receiver Operating Characteristic (ROC) curve is another commonly used method for evaluating classifier performance across different thresholds. It plots the true-positive rate against the false-positive rate (FPR):

$$\text{FPR} = \frac{FP}{FP + TN}, \tag{2.3}$$

where TN represents true negatives. In fraud detection, the number of legitimate transactions is typically much larger than the number of fraudulent transactions. This is also the case for the datasets used in this thesis, where legitimate transactions outnumber fraudulent ones by roughly 27:1 in the IEEE-CIS dataset and by more than 500:1 in the ULB dataset.

Because the FPR is calculated using the total number of negative transactions, a large number of false positives can still correspond to a relatively small FPR when the negative class is very large. For example, a model may generate thousands of false alarms while the FPR remains comparatively low. The ROC curve can therefore make the practical impact of false positives less visible in highly imbalanced settings.

This does not mean that ROC-AUC is necessarily high when the data are imbalanced. A model with little or no ability to distinguish between the two classes can still have an ROC-AUC close to 0.5. The issue is instead that the ROC representation can understate the practical significance of false positives when the number of legitimate transactions is very large.

The precision-recall curve provides a different perspective. Precision is defined as

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{2.4}$$

so false positives directly affect the denominator. When the number of false alarms increases without a corresponding increase in correctly detected fraud, precision decreases. Unlike the FPR, precision does not depend on the number of true negatives. This makes the precision-recall curve particularly useful when the positive class is rare and false positives are operationally important.

Saito and Rehmsmeier [18] discuss this difference and show that precision-recall curves can provide a more informative assessment of classifier performance when the positive class is highly imbalanced. For this reason, AUC-PR is used in this thesis as the main threshold-independent metric for comparing the ranking performance of the models. The final operating point is evaluated separately using precision, recall, raw error counts, and the cost-based objective.

2.4.5 Confusion Matrix and Raw Error Counts

In addition to the summary metrics, this thesis reports the raw numbers of false positives and false negatives (Section 5.5). These counts provide useful context that percentage-based metrics alone may not capture.

This is particularly relevant for the ULB dataset, where the number of fraudulent transactions in the test set is relatively small. As a result, changing the classification of only a small number of transactions can have a noticeable effect on measures such as recall. Reporting the underlying counts makes these changes easier to interpret and shows the actual number of fraudulent transactions missed as well as the number of legitimate transactions incorrectly flagged.

Raw error counts are also important for the cost-sensitive analysis because false positives and false negatives can have different associated costs. They therefore provide a direct connection between the standard evaluation metrics and the financial cost assigned to each type of classification error.

2.4.6 Cost-Based Evaluation

Finally, model performance is evaluated using the total cost associated with false positives and false negatives under an explicit, asymmetric cost matrix. This provides a direct measure of the financial consequences of the model's decisions and complements the statistical metrics described above.

The cost-based evaluation is particularly important because a model with better precision, recall, or AUC-PR is not necessarily the model that produces the lowest financial cost. The preferred model and operating threshold therefore depend not only on its statistical performance but also on the relative costs assigned to different types of errors.

The cost-based evaluation methodology is formalised in Chapter 3, and the corresponding experimental results are presented in Chapter 6.

3 Cost-Sensitive Machine Learning

Most machine learning classifiers are trained and evaluated as if every misclassification were equally costly. That assumption breaks down in fraud detection, where missing a fraudulent transaction (a false negative) and wrongly flagging a legitimate one (a false positive) carry very different financial and operational consequences. This chapter sets out the theory of cost-sensitive learning that the methods in Chapter 6 build on, following mainly the formal treatment of Elkan [10] and, for the fraud-specific cost values, the work of Bahnsen et al. [7] and Sahin et al. [17].

3.1 From Accuracy to Expected Loss

A standard classifier minimises the overall error rate, which treats a false negative and a false positive as equally bad. For fraud detection that is a poor stand-in for the real objective: missing a €1,000 fraudulent transaction hurts far more than needlessly reviewing a €5 legitimate one. Cost-sensitive learning replaces error minimisation with the minimisation of *expected loss*: each type of error is given a cost, and the classifier, or its decision rule, is tuned to minimise total expected cost rather than the number of mistakes.

Formally, for a decision i (the action taken, e.g., "block" or "approve") and a true class j ("fraud" or "legitimate"), the expected loss of choosing i for an example x is:

$$L(x, i) = \sum_{j} P(j \mid x)\, C(i, j) \tag{3.1}$$

where $P(j \mid x)$ is the model's estimated probability that x belongs to class j, and $C(i, j)$ is the cost of choosing i when the truth is j [10]. The best decision is not necessarily the one with the highest predicted probability. Instead, it is the decision that results in the lowest expected loss. Elkan also shows that the optimal decision does not change if a constant is added to all costs in the same column, or if all costs are multiplied by a positive value. Therefore, when using a fixed 2×2 cost matrix, the decision mainly depends on the relative cost of false positives compared with false negatives [10]. In practice, this means that the exact monetary values of the costs do not need to be known precisely. What matters most for determining the optimal threshold is their relative size.

3.2 The Cost Matrix

For binary classification the costs fit into a 2×2 matrix:

	Truth: Legitimate	Truth: Fraud
Predict Legitimate	c_{00} (true negative, cost 0)	c_{01} (false negative)
Predict Fraud	c_{10} (false positive)	c_{11} (true positive, e.g., review cost)

Table 3.1: General cost matrix for binary fraud classification.

In fraud detection the dominant entry is c_{01}, the cost of a missed fraud, since it stands for the money lost on the undetected transaction. The false-positive cost c_{10} is usually much smaller and captures operational overhead: the effort of a manual review, or the friction of wrongly declining a customer. By convention c_{00} is set to zero, while c_{11} may carry a small residual cost for investigating a correctly flagged fraud. [7]

3.2.1 Class-Dependent and Example-Dependent Costs

Two ways of filling in this matrix appear in the literature. In the *class-dependent* case, the cost of an error depends only on its type: every missed fraud is charged the same amount and every false alarm the same, so a single pair of numbers describes the whole dataset. This is the classical imbalance-aware setting, usually realised by weighting the minority class more heavily during training [7], [17].

The *example-dependent* case fits fraud more naturally, because the stakes differ from one transaction to the next, missing a €5 fraud is not the same as missing a €5,000 one (Section 5.2). Here the cost of a false negative is tied to the individual transaction. Bahnsen et al. take that cost to be the transaction *amount* and charge a fixed administrative cost C_a for every declined transaction [7]; Sahin et al. instead take it to be the card's available credit *limit* [17]. Both argue that a single constant cost cannot represent fraud losses and that accuracy- and AUC-based evaluation is therefore inappropriate. This thesis adopts the Bahnsen formulation, since the transaction amount is available in both datasets whereas a card-level credit limit is not; the resulting cost scenarios are defined in full in Chapter 6.

3.3 The Cost-Sensitive Optimal Threshold

A classifier that outputs a probability $P(\text{fraud} \mid x)$ is usually thresholded at 0.5 to produce a decision. That default is optimal only when the two errors cost the same [10]. The cost-sensitive threshold p^*, the probability at which it becomes rational to switch from predicting "legitimate" to "fraud", is given by Elkan [10] as:

$$p^* = \frac{c_{10} - c_{00}}{c_{10} - c_{00} + c_{01} - c_{11}} \tag{3.2}$$

Because a missed fraud (c_{01}) costs far more than a false alarm (c_{10}), p^* sits well below 0.5: it is worth flagging a transaction even on fairly weak evidence, since erring toward a false alarm is cheap next to erring toward a missed fraud. This is precisely what the baseline results in Section 5.5 lack: all four baselines, thresholded at 0.5 with no cost adjustment, reach high precision but capped recall, letting a large share of fraud through.

3.3.1 Worked Example: The Amount-Based Threshold

Substituting the example-dependent (Bahnsen) costs into Equation 3.2 makes the threshold concrete. With $c_{00} = 0$, $c_{10} = c_{11} = C_a$, and c_{01} equal to the transaction amount a, the general expression collapses to

$$p^*(x) = \frac{C_a - 0}{(C_a - 0) + (a - C_a)} = \frac{C_a}{a}, \tag{3.3}$$

so a transaction should be declined whenever $P(\text{fraud} \mid x) \geq C_a/a$. This per-transaction rule is the Bayes-minimum-risk decision of Bahnsen et al. [7].

To see how the threshold behaves, consider the primary administrative cost used in this thesis, $C_a = 2.50$ EUR, and three transactions of different value. Substituting into $p^*(x) = C_a/a$ gives

$$p^*(10) = \frac{2.50}{10} = 0.25, \qquad p^*(100) = \frac{2.50}{100} = 0.025, \qquad p^*(1000) = \frac{2.50}{1000} = 0.0025. \tag{3.4}$$

A €10 transaction is declined only once the model is at least 25% certain it is fraudulent, whereas a €1,000 transaction is declined at a mere 0.25% certainty. The threshold is thus inversely proportional to the amount at stake and falls steeply as that amount rises: the more money a fraud would cost, the weaker the evidence needed to justify blocking it. In the limit, any transaction whose amount does not exceed C_a yields $p^* \geq 1$ and is never worth declining, blocking it would cost more in administrative overhead than the fraud itself could lose. All three thresholds lie far below the default of 0.5, which already indicates why a cost-insensitive classifier thresholded at 0.5 misses so much fraud, and why the cost-optimal operating point must sit well below it. The behaviour of this rule, and its limitations relative to a single global threshold, are examined empirically in Chapter 6.

3.4 Implementation Strategies

There are two broad ways to incorporate cost sensitivity into a classification system: during model training or at the decision stage [10]. This thesis considers both approaches. Within training-time weighting, two variants are examined: class-level weighting and example-dependent sample weighting.

3.4.1 Loss Function Weighting (Training-Time)

The first approach incorporates costs directly into model training by modifying the learning objective. Rather than treating each transaction equally in the loss function, errors on minority or high-value transactions are penalised more heavily. This thesis examines two training-time weighting schemes:

- **Class-level weighting:** The loss contribution of the positive (fraud) class is scaled by the inverse class ratio $N_{\text{neg}}/N_{\text{pos}}$ (using `scale_pos_weight` in XGBoost and `class_weight` in the neural network), penalising missed fraud proportionally to its rarity.
- **Example-dependent sample weighting:** Following the Bahnsen cost formulation, each training instance is assigned an individual weight equal to its potential cost: the transaction amount a for fraudulent transactions, and the administrative cost C_a for legitimate transactions.

Because modifying the training objective alters the prior probability distribution learned by the classifier, predicted probabilities become distorted toward higher values [9]. Consequently, models trained with cost weights must undergo probability recalibration on validation data before decision-theoretic thresholds can be applied.

3.4.2 Threshold Moving

The second leaves training and the underlying probability estimates untouched and simply moves the decision threshold to p^* at prediction time. Elkan argues this is generally the better option [10], because it separates training from the cost structure: if the business costs change, say the assumed cost of a missed fraud is revised, the threshold can be updated without retraining the model. That flexibility suits the comparison in this thesis, where XGBoost and neural networks are evaluated under several cost scenarios (Section 6.1).

3.5 Algorithm-Specific Considerations

Model classes react differently to cost adjustments. For tree-based models, rebalancing the training set does little to how the trees are split, because the ordering of class probabilities

used to choose splits barely depends on class proportions [10]. Pruning is the weaker point: criteria based on overall error reduction can cut branches that mainly capture the rare fraud class. This matters for the XGBoost baseline in Section 5.5, where the tree-depth and minimum-child-weight settings (`max_depth`, `min_child_weight`) effectively govern how hard minority branches are pruned, and it is picked up again when the cost-sensitive XGBoost models are tuned in Chapter 6.

For classifiers whose predicted probabilities are poorly calibrated, scores pushed toward 0 or 1 even when the real uncertainty is higher, rebalancing is discouraged; the working threshold should instead be found empirically, by testing decisions against the assumed cost structure rather than trusting the nominal probabilities [10]. This applies to the neural networks used here, whose sigmoid outputs are not guaranteed to be calibrated [9], and it is why threshold choices are validated against the cost-based metrics of Section 6.1 rather than applying p^* blindly.

3.6 Summary

This framework reframes the question the thesis asks. Instead of which model reaches the highest accuracy, or even the highest recall on its own, the question becomes which model and which cost-sensitive strategy minimise total expected financial loss under a realistic cost structure for false negatives and false positives. Chapter 6 puts this to work: it defines cost scenarios for both datasets and evaluates XGBoost and neural networks under both the rebalancing and threshold-moving strategies introduced here.

4 Machine Learning Models

This chapter introduces the two model families evaluated throughout this thesis: XGBoost, an ensemble method based on gradient-boosted decision trees, and a feedforward artificial neural network. Both represent standard architectures for tabular classification, yet they possess fundamentally different inductive biases, feature representations, and baseline responses to class imbalance. To isolate their performance, both models were trained and evaluated under a unified preprocessing workflow (Chapter 5), with model-specific feature encoding introduced only where structurally required.

4.1 XGBoost

4.1.1 Gradient Boosting Principles

XGBoost (Extreme Gradient Boosting) [6] is an ensemble technique that constructs decision trees sequentially, with each successive tree trained to approximate the negative gradients (pseudo-residuals) of the objective function evaluated at the current ensemble prediction. For a dataset comprising n observations and m features, the ensemble prediction $\hat{y}_i$ is the additive sum of outputs across K base regression trees:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i), \quad f_k \in \mathcal{F} \tag{4.1}$$

where $\mathcal{F} = \{f(x) = w_{q(x)}\}$ denotes the hypothesis space of classification and regression trees (CART), $q : \mathbb{R}^m \to T$ maps an input vector to a leaf index, and $w \in \mathbb{R}^T$ represents the corresponding vector of leaf weights. Trees are added greedily by minimising a second-order Taylor expansion of the loss function regularised by a structural penalty:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^{T} w_j^2 \tag{4.2}$$

where g_i and h_i represent the first- and second-order loss gradients with respect to the current prediction, T denotes the total number of terminal leaves, and γ and λ penalise leaf count and weight magnitudes, respectively. This structural regularisation prevents over-parameterisation on high-dimensional tabular datasets such as IEEE-CIS, which comprises 394 raw transaction columns and expands to 415 features post-merging and encoding.

4.1.2 Sparsity-Aware Split Finding

A central algorithmic property of XGBoost is its native handling of missing values. Its sparsity-aware split-finding algorithm evaluates candidate branch directions exclusively over non-missing observations and assigns a *default direction* to each split node based on maximal gradient gain [6]. Missing entries encountered at inference time follow this learned default routing. While classical decision trees often rely on surrogate splits [5], XGBoost optimizes split gain directly over incomplete observations. In this thesis, to maintain strict input parity across model families, both XGBoost and the neural network were evaluated on the median-imputed feature matrix (Section 5.3); nevertheless, XGBoost's structural insensitivity to missingness boundaries prevents arbitrary imputation constants from distorting split gains.

4.1.3 Suitability for Tabular Fraud Detection

Gradient-boosted trees remain a dominant standard for credit card fraud detection [2]. Their recursive, axis-aligned partitions naturally capture high-order feature interactions, such as joint dependencies between card types, transaction amounts, and merchant categories, without requiring explicit interaction engineering. Furthermore, XGBoost yields raw continuous margins that map to well-behaved posterior probabilities via the logistic link function, facilitating the post-hoc decision-theoretic thresholding strategies formulated in Chapter 3.

4.2 Neural Networks for Tabular Data

4.2.1 Architecture

Feedforward artificial neural networks applied to tabular tasks typically consist of stacked fully connected (dense) layers. For a layer parameterised by weight matrix W, bias vector b, and activation function σ, the hidden activation vector h for input x is:

$$h = \sigma(Wx + b) \tag{4.3}$$

The neural network baseline (Section 5.4) uses a compact multilayer perceptron topology: two hidden layers containing 64 and 32 units with Rectified Linear Unit (ReLU) activations, each followed by a dropout layer ($p = 0.3$) to limit co-adaptation. The output layer consists of a single neuron parameterised by a sigmoid activation $\sigma(z) = (1 + e^{-z})^{-1}$, generating a continuous posterior probability estimate $\hat{p} \in [0, 1]$.

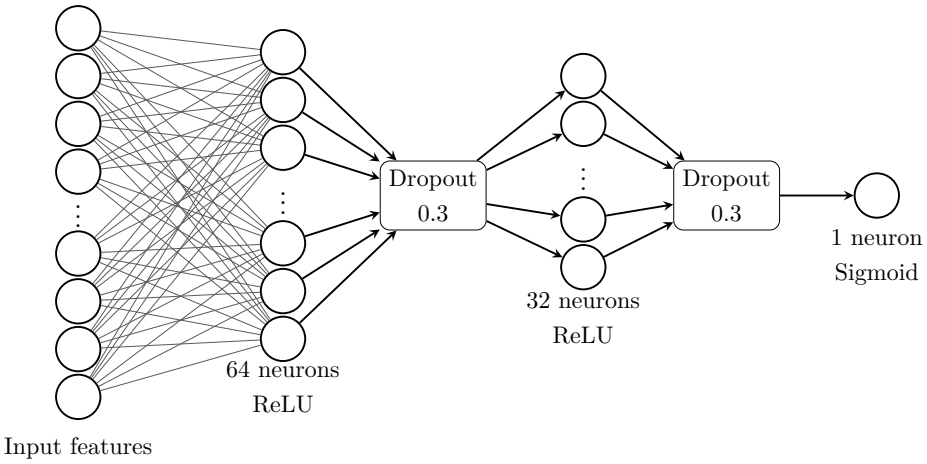


Figure 4.1: Baseline neural network architecture for IEEE-CIS fraud detection. Representative neurons are shown; dots indicate omitted neurons.

The narrowing $64 \to 32 \to 1$ topology implements a conservative funnel that compresses high-dimensional inputs toward the scalar decision output. Intermediate dropout ($p = 0.3$) provides stochastic regularisation against feature co-adaptation, which is critical given the known sensitivity of neural networks to uninformative features in tabular settings [11]. The architecture was held fixed across all experiments, as detailed in Section 4.3.2: maintaining a stable, unvaried parametric baseline ensures that downstream differences in performance reflect the cost-sensitive interventions rather than changes in network topology.

4.2.2 Data Requirements

In contrast to tree ensembles, neural networks require dense, strictly numerical representations and are sensitive to feature magnitude. Gradient descent optimisation exhibits poor numerical stability when input features span divergent ranges, requiring continuous variables to be standardised ($\mu = 0, \sigma = 1$) using parameters fitted exclusively on the training partition. While tree ensembles are theoretically invariant to monotonic feature scaling, continuous variables were standardised across both models to preserve input parity. Furthermore, while low-cardinality categorical features were one-hot encoded identically for both architectures, the five high-cardinality categorical variables on IEEE-CIS (`P_emaildomain`, `DeviceInfo`, and device identity columns) required numerical conversion for the neural network: they were frequency-encoded on the training split, whereas XGBoost processed them via native categorical partitioning.

4.2.3 Baseline Behavior Under Extreme Imbalance

When trained under unweighted binary cross-entropy on datasets with extreme class skew, gradient descent updates are dominated by majority-class errors. Without cost-sensitive

loss weighting, the network's predictive distribution collapses toward low probabilities. This empirical limitation is visible in the baseline evaluation (Section 5.5), where the unweighted neural network achieves a fraud recall of only 0.336 on IEEE-CIS at the conventional threshold ($t = 0.5$).

4.3 Hyperparameter Tuning and Training Protocol

The two model families were configured according to their structural characteristics. XGBoost exposes several influential hyperparameters governing tree complexity and regularisation, so its configuration was selected via randomized search on the training partition; the neural network utilized a fixed reference architecture, motivated in Section 4.3.2. In both cases, model tuning and validation strictly excluded the held-out test set, preserving an unbiased final evaluation.

4.3.1 XGBoost

XGBoost was tuned using randomized search (`RandomizedSearchCV`) [15], which samples a predetermined budget of configurations from a parameter space rather than conducting an exhaustive grid search. Randomized search was selected because it identifies high-performing configurations at a fraction of the computational expense when a small subset of hyperparameters dominates empirical performance [4]. Thirty candidate configurations were sampled from the search space defined in Table 4.1, with each configuration evaluated using three-fold stratified cross-validation on the training set.

Crucially, candidate configurations were scored by the area under the precision–recall curve (PR-AUC), rather than accuracy or ROC-AUC. Given the severe class imbalance in both datasets, PR-AUC captures performance on the rare fraud class directly, whereas accuracy and ROC-AUC are heavily distorted by the overwhelming number of legitimate transactions [18]. Direct optimization on PR-AUC aligns model selection with the fraud interception objective.

Table 4.1: Search space for the XGBoost randomized search (30 iterations, three-fold stratified cross-validation, scored on PR-AUC).

Hyperparameter	**Candidate values**
`n_estimators`	100, 200, 300, 500
`max_depth`	3, 4, 5, 6, 8
`learning_rate`	0.01, 0.05, 0.1, 0.2, 0.3
`subsample`	0.6, 0.8, 1.0
`colsample_bytree`	0.6, 0.8, 1.0
`min_child_weight`	1, 3, 5

The search was executed independently on each dataset to account for differences in feature dimension and noise characteristics. The resulting configurations are reported in Table 4.2. On the high-dimensional, heterogeneous IEEE-CIS dataset, the search favored deeper trees (`max_depth` = 8) with a relatively aggressive learning rate (0.3) and feature subsampling (`colsample_bytree` = 0.6). Conversely, on the lower-dimensional, PCA-transformed ULB dataset, the search selected shallower trees (`max_depth` = 4) paired with a conservative learning rate (0.05) and sample subsampling (0.8), producing a model less susceptible to overfitting the compact feature space.

Table 4.2: Best XGBoost configurations selected by randomized search, per dataset, with corresponding cross-validated PR-AUC.

Hyperparameter	**IEEE-CIS**	**ULB**
`n_estimators`	200	300
`max_depth`	8	4
`learning_rate`	0.3	0.05
`subsample`	1.0	0.8
`colsample_bytree`	0.6	0.8
`min_child_weight`	1	1
CV PR-AUC	0.753	0.838

4.3.2 Neural Network

The neural network was not tuned using an extensive hyperparameter search. Instead, it utilized the fixed baseline architecture introduced in Section 4.2: a multilayer perceptron comprising two hidden layers of 64 and 32 units with ReLU activations, intermediate dropout layers with a rate of 0.3, and a single sigmoid output neuron. The network was

trained using the Adam optimiser (initial learning rate $\eta = 10^{-3}$) and binary cross-entropy loss for 20 epochs with a mini-batch size of 256 and an internal 10% validation split for training monitoring.

This setup was a deliberate experimental control rather than an omission. The core objective of this research is to isolate the empirical effect of cost-sensitive interventions. Extensive architecture and topology searches for the neural network would introduce additional confounding factors, as observed performance shifts could then reflect specific architectural hyperparameter choices rather than the cost-weighting and decision-theoretic mechanisms under study. To ensure reproducibility across all experiments, random seeds for dataset partitioning, XGBoost stochastic sampling, and neural network weight initialisation were fixed to 42.

4.3.3 A Single, Fixed Pipeline

Following hyperparameter determination, each model configuration was frozen and evaluated without downstream architectural modifications. Both benchmark datasets passed through an identical, leak-free preprocessing workflow (Chapter 5): stratified partitioning strictly preceded all transformations, missing numeric values were imputed using training medians exclusively, and feature scaling was fitted on the training split alone. Because the baseline classifiers and their cost-sensitive counterparts share this common pipeline, observed variations in detection performance and financial loss are directly attributable to the cost-sensitive mechanisms rather than discrepancies in data representation or information leakage.

4.4 Comparison: Strengths and Weaknesses for Tabular Fraud Detection

XGBoost and the feedforward neural network exhibit distinct inductive biases when processing structured tabular transaction data. Gradient-boosted decision trees partition feature space via axis-aligned splits, demonstrating robustness to unscaled numeric ranges and differing feature distributions. In contrast, neural networks optimize continuous loss functions through gradient backpropagation, necessitating smooth numeric representations and standardized inputs. Table 4.3 summarizes the two model families across operational dimensions relevant to fraud detection.

These structural differences explain the baseline results obtained prior to any cost-sensitive adjustments (Section 5.5). Under standard unweighted objectives and evaluated at the conventional decision threshold ($t = 0.5$), XGBoost achieved a fraud recall of 0.602 on the heterogeneous, high-dimensional IEEE-CIS dataset, compared to 0.336 for the neural network. On the clean, lower-dimensional, PCA-transformed ULB dataset, this disparity

Dimension	**XGBoost**	**Neural Network**
Missing data	Robust to uninformative missingness; supports default branch routing [6]	Requires numerical imputation prior to forward-pass computation
Categorical inputs	Handled via one-hot encoding and native category splits	Requires numerical conversion (one-hot and frequency encoding)
Feature scaling	Invariant to strictly monotonic transformations	Requires standardisation ($\mu = 0, \sigma = 1$) for gradient stability
Class imbalance	Addressed via loss weighting (`scale_pos_weight`) and thresholding	Heavily biased toward majority class under unweighted cross-entropy
Interpretability	Direct tree split metrics (gain, weight, coverage)	Requires post-hoc attribution methods
Preprocessing burden	Moderate: robust to raw feature distributions	High: strict requirement for complete imputation and scaling

Table 4.3: Comparison of XGBoost and neural network properties in tabular fraud detection.

contracted (0.747 versus 0.716). These results are consistent with broader empirical literature on tabular deep learning [11], establishing that tree ensembles maintain superior inductive efficiency when processing uncurated tabular data characterized by severe class imbalance.

5 Data and Methodology

This chapter describes the concrete implementation of the data pipeline and baseline models used in this thesis. It covers dataset preparation, the missing-data methodology developed for the IEEE-CIS dataset, the feature engineering and leakage-aware preprocessing pipeline, and the results of baseline XGBoost and Neural Network classifiers trained without cost-sensitive adjustments. These baseline results establish the reference point against which the cost-sensitive methods introduced in Chapter 6 are evaluated.

5.1 Datasets

Two publicly available datasets were selected for their complementary characteristics, allowing the generalizability of cost-sensitive learning strategies to be tested across markedly different data conditions.

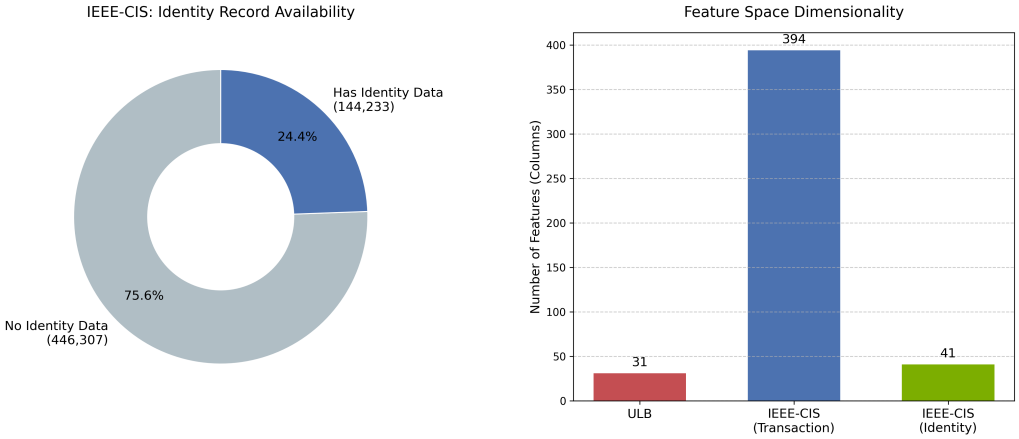


Figure 5.1: Structure of the datasets. Left: identity-record availability in IEEE-CIS. Right: feature-space dimensionality of the ULB, IEEE-CIS transaction, and IEEE-CIS identity tables.

5.1.1 ULB Credit Card Fraud Dataset

The ULB dataset [13] contains 284,807 transactions made by European cardholders over two days in September 2013, of which 492 are confirmed fraud cases, corresponding to a fraud rate of 0.172%. The dataset consists of 31 columns: `Time`, `Amount`, the binary label

`Class`, and 28 principal components (`V1`–`V28`) obtained via PCA transformation of the original features, which are not disclosed for confidentiality reasons. `Time` and `Amount` are the only interpretable features. During exploratory analysis, 1,081 duplicate rows were identified and confirmed to be data artifacts; these were removed, reducing the working dataset to 283,726 rows.

5.1.2 IEEE-CIS Fraud Detection Dataset

The IEEE-CIS dataset [20], provided by Vesta Corporation for a Kaggle competition hosted by the IEEE Computational Intelligence Society, contains 590,540 transactions with 20,663 confirmed fraud cases (3.5% fraud rate). The data is split across two files joined on `TransactionID`:

- `train_transaction.csv`, 590,540 rows, 394 columns, including the `isFraud` label, transaction amount, product category, card information, and anonymized `V1-V339` features.
- `train_identity.csv`, 144,233 rows, 41 columns of device and network metadata. Only approximately 24.4% of transactions have an associated identity record.

The competition additionally provides a separate test set (`test_transaction.csv` / `test_identity.csv`); however, its `isFraud` labels are withheld for competition scoring and are therefore unavailable for offline evaluation. All modelling and evaluation in this thesis consequently use only the labelled `train_transaction.csv`, partitioned into training, validation, and test sets via stratified sampling (Section 5.4). Compared to ULB, IEEE-CIS offers a moderate class imbalance (3.5% vs. 0.172%), a substantially larger number of fraud cases (20,663 vs. 492), and a richer, partially interpretable feature set, at the cost of extensive missing data.

5.2 Exploratory Data Analysis

Exploratory analysis of IEEE-CIS revealed several fraud-relevant patterns that directly informed the preprocessing and feature engineering decisions described in the following sections. These associations are univariate and unadjusted, and are reported here as indications of where predictive signal appears rather than as causal claims.

- **Identity presence.** Transactions with an associated identity record show a fraud rate of 7.85%, compared to 2.09% for transactions without one. A plausible interpretation is that identity metadata (device type, browser, screen resolution) is captured predominantly for online, card-not-present transactions, so the presence of an identity record acts as a proxy for the transaction channel; the anonymized data does not permit direct verification of this hypothesis. Regardless of interpretation, the mere presence or absence of an identity record is itself a strong predictive signal,

which motivates the engineered `has_identity` feature described in Section 5.4.

- **Email domain.** `protonmail.com` shows a fraud rate of 40.8%, the strongest single interpretable predictor identified in the dataset. Mainstream providers such as `gmail.com` (4.3%) and `yahoo.fr` (3.5%) show markedly lower rates.
- **Product category.** Product `C` shows an 11.7% fraud rate, roughly three times the dataset average, while product `W` shows the lowest rate at 2.0%.
- **Card type.** Credit cards show a 6.7% fraud rate versus 2.4% for debit cards.
- **Device type.** Mobile transactions show a higher fraud rate (10.2%) than desktop transactions (6.5%).

The categorical fraud-rate patterns above are summarised in Figure 5.2, and the email-domain effect in Figure 5.3.

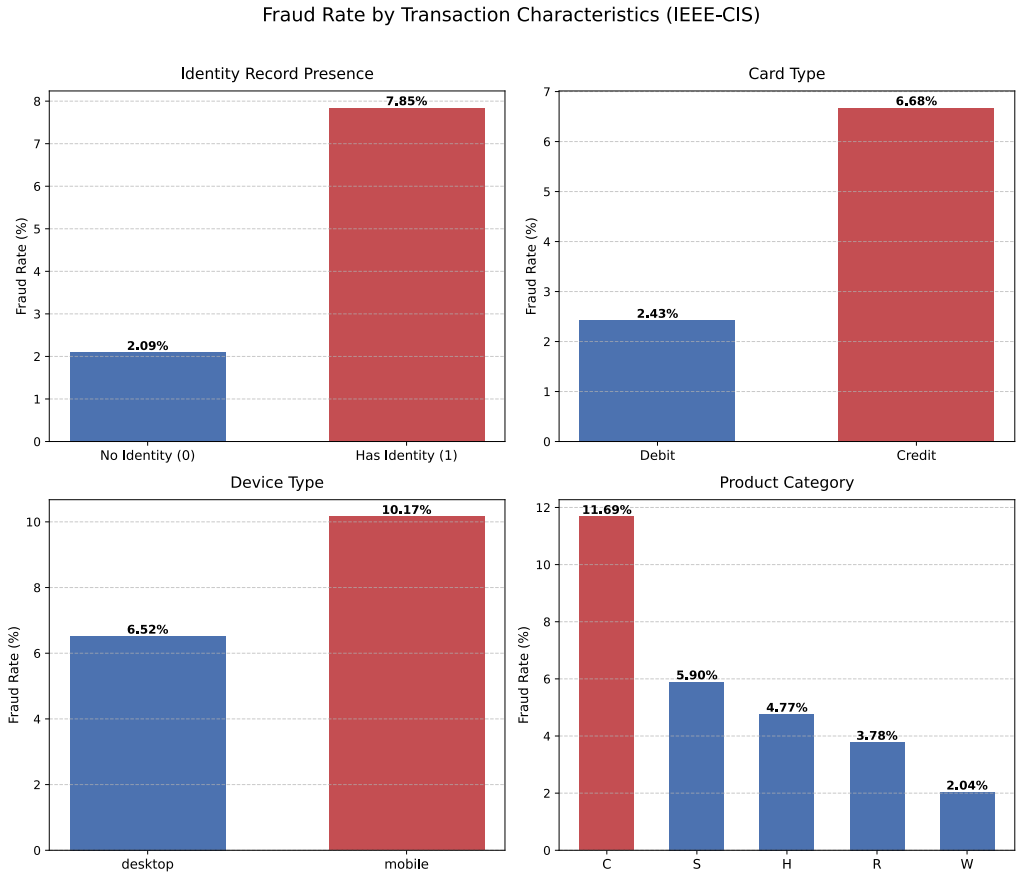


Figure 5.2: Fraud rate by transaction characteristic in IEEE-CIS (merged transaction and identity data): product category, card type, device type, and the presence of an identity record. Bars show the univariate fraud rate within each category against the dataset baseline.

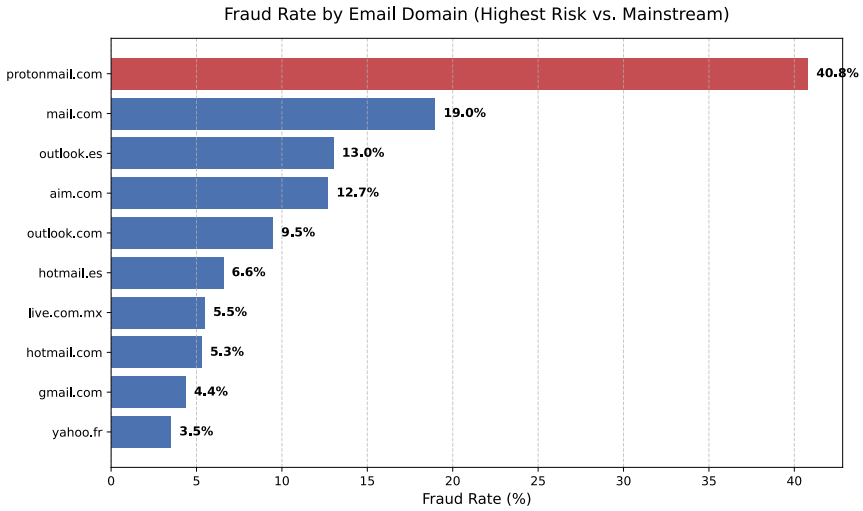


Figure 5.3: Fraud rate by purchaser email domain in IEEE-CIS, restricted to domains with at least 50 transactions. `protonmail.com` is the single strongest interpretable predictor identified, far above mainstream providers such as `gmail.com` and `yahoo.fr`.

Transaction amounts differ between fraudulent and legitimate transactions, though the pattern is not identical across the two datasets. In both, fraudulent transactions have a higher *mean* amount than the overall population (ULB: €122.2 versus €88.3; IEEE-CIS: €149.2 versus €135.0). Their medians, however, diverge: on ULB the fraud median is markedly *lower* than the population median (€9.25 versus €22.0), indicating that ULB fraud is dominated by many small-value transactions alongside a smaller number of high-value outliers, whereas on IEEE-CIS the fraud and overall medians are close (€75.0 versus €68.8). In both cases the mean exceeds the median for fraud, confirming a right-skewed amount distribution with a long high-value tail. This variation in the value at stake from one transaction to the next directly motivates the example-dependent cost weighting explored in Chapter 3, since a fixed misclassification cost would treat a missed €5 fraud the same as a missed €5,000 fraud. [7]

5.3 Missing Data Methodology

The ULB dataset contains no missing entries and required no imputation. Conversely, IEEE-CIS exhibits pervasive missingness: 174 of the 394 features in `train_transaction.csv` exceed 50% missingness.

Table 5.1: Strongest informative-missingness signals among the 174 high-missingness columns in `train_transaction.csv`. Fraud prevalence is contrasted between missing and present observations against baseline prevalence.

Feature	Fraud rate (missing)	Fraud rate (present)	Relative gap
D7	2.70 %	14.88 %	3.48
D12	2.48 %	11.74 %	2.65
D14	2.55 %	11.60 %	2.59
D13	2.62 %	11.04 %	2.41
D6	2.50 %	10.55 %	2.30

5.3.1 Limitations of Proportion-Based Thresholds

A common heuristic in tabular machine learning workflows is to drop features exceeding a fixed missingness proportion, such as 50%. However, statistical missing-data theory cautions that the raw proportion of missing values should not by itself dictate handling decisions [14].

As demonstrated by Madley-Dowd et al. [14], the raw missingness proportion is an unreliable indicator of actual information loss because it fails to account for the predictive capacity of auxiliary variables in the dataset. By evaluating the Fraction of Missing Information (FMI), which quantifies parameter uncertainty while accounting for the recovery power of correlated predictors, they establish that high missingness (even exceeding 80–90%) does not preclude valid statistical estimation when informative correlates exist.

Translating this principle to tabular feature selection, dropping candidate predictors strictly based on an arbitrary missingness threshold risks discarding valuable diagnostic signal. In financial fraud detection, severe sparsity is rarely missing completely at random (MCAR); instead, whether a value is observed or absent often correlates directly with the transaction channel, device telemetry, or fraudulent evasion behavior.

5.3.2 Gap-Based Missingness Check

Following this reasoning, each of the 174 high-missingness columns in `train_transaction.csv` was tested for an association between missingness and the fraud label, rather than being dropped by proportion alone. This is consistent with established practice for assessing missingness mechanisms, in which the probability of a value being missing is modelled as a function of observed variables to determine whether specific observable characteristics are systematically associated with missingness [14]. For each column, the fraud rate among transactions with a missing value was compared to the fraud rate among transactions with a present value, and the relative gap between the two was computed against the dataset's

baseline fraud rate:

$$\text{relative_gap} = \frac{|\text{fraud_rate}_{\text{missing}} - \text{fraud_rate}_{\text{present}}|}{\text{fraud_rate}_{\text{baseline}}} \tag{5.1}$$

```
base_rate = df['isFraud'].mean()
for col in high_missing_cols:            # columns with > 50% missing
    rate_missing = df.loc[df[col].isna(),  'isFraud'].mean()
    rate_present = df.loc[df[col].notna(), 'isFraud'].mean()
    relative_gap = abs(rate_missing - rate_present) / base_rate
    # flag column if relative_gap >= 1.0, else drop it cleanly
```

Listing 5.1: Gap-based informative-missingness test: for each high-missingness column, the fraud rate among missing versus present values is compared against the baseline fraud rate.

This mirrors the approach of [14], who report an odds ratio for each auxiliary variable quantifying whether it predicts missingness in the outcome; the relative-gap statistic used here is the binary-outcome analogue of that odds-ratio comparison.

5.3.3 Results and Threshold Selection

The 174 candidate columns separated into three tiers by relative gap:

Tier	Relative gap	Approx. column count
Weak / no signal	0.11 – 0.32	53
Moderate signal	0.72 – 1.96	114
Strong signal	2.27 – 3.48	7

Table 5.2: Distribution of relative missingness gap across the 174 high-missingness columns in `train_transaction.csv`.

The strongest effect was found in the `D`-block of time-delta features: `D7` shows a fraud rate of 2.7% when missing versus 14.9% when present, a relative gap of 3.48. A threshold of relative gap ≥ 1.0 was adopted: columns meeting this criterion received a binary indicator feature before the original column was dropped, while columns below the threshold were dropped without a replacement flag. This yielded 121 flagged columns and 53 cleanly dropped columns. The threshold was tested for stability against 0.5 and 2.0: results at 0.5 differed by only 4 of 174 columns from the 1.0 threshold, confirming stability in this range, while 2.0 was rejected as overly conservative, since it would have discarded clearly informative feature blocks (relative gap ≈ 1.6). This treatment is consistent with the broader tree-based missing-data literature: discarding features or observations with missing values is justified only for relatively small amounts of missing data under restrictive conditions [5], implying it is not automatically appropriate once missingness becomes large or structured. Retaining missingness as an engineered indicator feature, rather than

discarding it, provided a lightweight alternative to full multiple imputation, which was judged impractical given the scale of the dataset.

5.3.4 Differentiated Handling for Categorical Features

While all numeric columns were completely median-imputed across splits, categorical features received differentiated representation. Low-cardinality categoricals were one-hot encoded for both models. For the five high-cardinality categorical columns not covered by one-hot encoding (`P_emaildomain`, `id_30`, `id_31`, `id_33`, `DeviceInfo`), XGBoost leveraged native categorical split finding (`enable_categorical=True`) on pandas categorical dtypes. The Neural Network pipeline, which requires numeric representations, converted these five columns via frequency encoding learned exclusively from the training partition.

5.4 Preprocessing Pipeline

The complete end-to-end workflow from raw data ingestion to evaluation is illustrated in Figure 5.4. The sequential implementation details are described below.

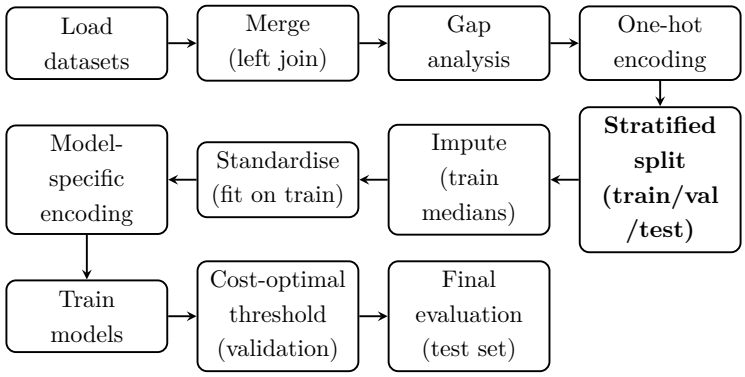


Figure 5.4: Leakage-aware preprocessing and evaluation pipeline. Transformations are fitted exclusively on the training split, thresholds are tuned on validation data, and final metrics are evaluated on the test partition.

5.4.1 Merging

`train_transaction.csv` and `train_identity.csv` were joined on `TransactionID` using a left join, preserving all 590,540 transactions. Identity columns are populated with missing values for the approximately 75.6% of transactions without an identity record.

5.4.2 Feature Engineering

Beyond the missingness indicators introduced in Section 5.3, one additional structural feature was engineered directly from the merge topology. As established in Section 5.2,

the presence of an identity record is strongly correlated with fraudulent activity (7.85% versus 2.09%), indicating that missingness is not missing at random (MNAR) [14]. To make this diagnostic signal explicitly accessible to both classifiers, a binary indicator, `has_identity`, was constructed to record whether an identity record exists (1) or not (0). Crucially, this indicator was generated immediately after the join and *prior* to numerical and constant imputation, ensuring that downstream token fills do not obscure the record presence signal.

5.4.3 One-Hot Encoding of Low-Cardinality Categoricals

Feature cardinality was evaluated to systematically separate low-cardinality nominal features from continuous variables and high-cardinality metadata. A threshold of ten or fewer unique values was applied to identify categorical features suitable for dummy variable expansion.

For the ULB dataset, all 30 predictive features are numeric PCA components and transaction timestamps, resulting in zero categorical columns and requiring no encoding.

```
# Evaluate cardinality on merged IEEE-CIS dataset
cardinality_summary = df.nunique()
low_card_cols = cardinality_summary[cardinality_summary <= 10].index.
    tolist()

numeric_cols = [c for c in low_card_cols if df[c].dtype in ['float64', '
    int64']]
categorical_cols = [c for c in low_card_cols if df[c].dtype == 'object']
# Output: Numeric: 208 | Categorical (needs encoding): 19

# ULB dataset check
cardinality_summary_ulm = df_ulm.nunique()
low_card_cols_ulm = cardinality_summary_ulm[cardinality_summary_ulm <=
    10].index.tolist()
categorical_cols_ulm = [c for c in low_card_cols_ulm if df_ulm[c].dtype
    == 'object']
# Output: Categorical (needs encoding): 0
```

Listing 5.2: Evaluation of feature cardinality and identification of categorical columns.

On IEEE-CIS, exactly 19 low-cardinality categorical features were detected:

> `ProductCD`, `card4`, `card6`, `M1`, `M2`, `M3`, `M4`, `M6`, `id_12`, `id_15`, `id_16`, `id_28`, `id_29`, `id_34`, `id_35`, `id_36`, `id_37`, `id_38`, and `DeviceType`.

Prior to indicator expansion, missing values in these categorical fields were filled with a static `'Unknown'` category token. Because this constant assignment relies neither on target labels nor on sample distribution statistics, it introduces no information leakage and safely precedes dataset partitioning.

The 19 columns were expanded using full dummy indicators (`drop_first=False`). This expanded the IEEE-CIS feature dimension from 369 to 417 columns before splitting, ensuring identical column alignments across all subsequent training, validation, and test subsets.

```
# 1. Constant imputation for categoricals (leak-free)
cat_cols_all = df.select_dtypes(include=['object']).columns
df[cat_cols_all] = df[cat_cols_all].fillna('Unknown')

# 2. One-hot encode the 19 identified low-cardinality features
# Expands feature space from 369 to 417 columns
df = pd.get_dummies(df, columns=categorical_cols, drop_first=False)
```

Listing 5.3: Categorical imputation and one-hot encoding prior to partitioning.

The remaining five high-cardinality categorical attributes (`P_emaildomain`, `id_30`, `id_31`, `id_33`, and `DeviceInfo`) were purposefully excluded from one-hot dummy expansion to avoid severe feature sparsity. Instead, they were preserved for model-specific post-split handling: native category partitioning for XGBoost, and frequency encoding for the neural network (Section 5.4.5).

5.4.4 Train/Validation/Test Split

Both datasets were partitioned via stratified sampling to preserve the empirical fraud ratio across all subsets. An initial stratified 80/20 split isolated the held-out test partition (118,108 transactions for IEEE-CIS; 56,746 for ULB following artifact deduplication). The remaining 80% of the data was partitioned again via an 80/20 stratified split into training and validation sets, yielding an overall 64/16/20 dataset partitioning. The validation set was reserved strictly for probability recalibration and cost-optimal threshold tuning (Chapter 6); the test set was evaluated once for final metric reporting. All partitioning strictly preceded numeric median imputation, continuous scaling, and frequency encoding (Listing 5.4).

```
# 1. Split BEFORE any fitting (3-way, stratified, seeded)
X_trainval, X_test, y_trainval, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(
    X_trainval, y_trainval, test_size=0.20,
    stratify=y_trainval, random_state=42)

# 2. Impute with TRAIN medians only, apply to all splits
num_cols = X_train.select_dtypes(include=['float64', 'int64']).columns
train_medians = X_train[num_cols].median()
X_train[num_cols] = X_train[num_cols].fillna(train_medians)
X_val[num_cols]   = X_val[num_cols].fillna(train_medians)
```

```
X_test[num_cols]  = X_test[num_cols].fillna(train_medians)

# 3. Scale: fit on TRAIN only, apply to all splits
scaler = StandardScaler().fit(X_train[scale_cols])
X_train[scale_cols] = scaler.transform(X_train[scale_cols])
X_val[scale_cols]   = scaler.transform(X_val[scale_cols])
X_test[scale_cols]  = scaler.transform(X_test[scale_cols])
```

Listing 5.4: Leakage-aware pipeline: the split precedes all fitting; imputation medians and the scaler are learned on the training partition only and applied unchanged to validation and test.

5.4.5 Leakage-Aware Imputation, Scaling, and High-Cardinality Encoding

Any transformation that computes sample statistics must be fitted exclusively on the training partition to prevent optimistic evaluation bias caused by data leakage. Missing values in numeric features were replaced with the column medians computed on X_{train} and applied unchanged to the validation and test sets [5]. All continuous variables were then standardised ($\mu = 0, \sigma = 1$) using a `StandardScaler` fitted only on the training split. While tree-based ensembles are mathematically invariant to monotonic feature scaling, the standardized features were passed to both models to maintain identical feature matrices.

The five high-cardinality categorical features were processed according to each model's requirements. For XGBoost, they were retained as native pandas categoricals to leverage native split-finding (`enable_categorical=True`). For the neural network, which requires numeric representations, they were frequency-encoded using category counts calculated strictly on the training partition.

5.5 Baseline Modelling

To establish an untreated reference point, baseline XGBoost and neural network classifiers were trained without class weighting, custom loss adjustments, or resampling methods like SMOTE. These baselines serve as the benchmark for the cost-sensitive interventions evaluated in Chapter 6.

5.5.1 Setup

Hyperparameters for XGBoost were tuned using randomized search (`RandomizedSearchCV`, 30 iterations, 3-fold stratified cross-validation on the training set) scored on the area under the precision–recall curve (PR-AUC). PR-AUC was preferred over accuracy and ROC-AUC because it directly reflects performance on the rare fraud class without distortion from the large majority class [12]. The neural network was trained using the fixed

$64 \rightarrow 32 \rightarrow 1$ architecture (dropout rate 0.3) described in Chapter 4. Both models were evaluated on the held-out test partition at the conventional decision boundary ($t = 0.5$). Training was conducted on an NVIDIA Tesla P100 GPU.

5.5.2 Results

Metric	IEEE-CIS XGB	IEEE-CIS NN	ULB XGB	ULB NN
Precision	0.922	0.844	0.986	0.907
Recall	0.602	0.336	0.747	0.716
F1-score	0.728	0.481	0.850	0.800
PR-AUC	0.797	0.552	0.832	0.810

Table 5.3: Baseline classification performance evaluated at $t = 0.5$ without cost-sensitive adjustments on the held-out test partitions.

On the IEEE-CIS test set (4,133 confirmed fraud cases), XGBoost missed 1,647 frauds as false negatives and produced 209 false positives. The neural network missed 2,743 frauds while producing 257 false positives. On the ULB test set (95 fraud cases), XGBoost produced 24 false negatives and 1 false positive, whereas the neural network produced 27 false negatives and 7 false positives.

5.5.3 Discussion of Baseline Results

The baseline metrics illustrate how tree ensembles and neural networks behave differently across distinct tabular structures:

- **Performance on complex tabular features:** On IEEE-CIS, XGBoost substantially outperformed the neural network in fraud recall (0.602 vs. 0.336). Tree-based algorithms isolate rare subsets more effectively in irregular tabular data with mixed distributions and high-cardinality metadata [11]. In contrast, standard binary cross-entropy in the neural network is overwhelmed by legitimate transactions, pulling predicted probabilities toward zero.
- **Parity on clean continuous data:** On ULB, where features are dense, fully continuous, and decorrelated via PCA, the performance gap between both models narrowed considerably (recall of 0.747 vs. 0.716; PR-AUC of 0.832 vs. 0.810). This suggests neural networks remain competitive on smooth tabular spaces, but degrade faster on raw, heterogeneous inputs.
- **Sample size considerations:** Because the ULB test partition contains only 95 fraud cases, a single misclassified transaction shifts recall by roughly 1.05 percentage points. The higher absolute metrics on ULB should therefore be evaluated with this sensitivity in mind.

At the default threshold of $t = 0.5$, all baseline models show high precision but limited recall. A substantial portion of fraud volume goes undetected simply because standard objectives treat misclassifications symmetrically, motivating the cost-sensitive weighting and threshold tuning explored in the next chapter.

6 Cost-Sensitive Results

This chapter reports the empirical evaluation of the cost-sensitive strategies formulated in Chapter 2, measuring their performance against the cost-insensitive baselines established in Section 5.5. Under the decision-theoretic framework of Elkan [10] and Bahnsen et al. [7], classifiers are evaluated not by symmetric accuracy, but by the minimization of total expected financial loss across all transaction decisions.

Section 6.1 formalises the experimental protocol, the evaluated model variants, and the financial metrics. Section 6.2 presents the test findings, tracing the progression from the initial savings-driven approach to the refined global threshold (t^*) and the administrative cost sweep (C_a). Sections 6.3 and 6.4 analyze the architectural disparities between tree ensembles and neural networks and discuss dataset-specific operational implications.

6.1 Experimental Protocol and Cost-Based Metrics

The evaluation examines three model training variants across three decision rules and three administrative review costs ($C_a \in \{1.00, 2.50, 5.00\}$ EUR), computed exclusively on the held-out test partitions.

6.1.1 Evaluated Model Configurations

For both XGBoost and the neural network, three distinct training configurations are analyzed:

1. **Baseline:** Standard unweighted training under binary cross-entropy loss, evaluated at the default decision boundary ($t = 0.5$) as established in Section 5.5.
2. **Class-weighted (Scenario A):** Training-time loss weighting using the empirical inverse class ratio $N_{\text{neg}}/N_{\text{pos}}$ (`scale_pos_weight` in XGBoost, `class_weight` in Keras), penalising missed fraud proportionally to class rarity.
3. **Amount-weighted (Scenario B):** Example-dependent sample weighting during training. Following the asymmetric liabilities defined in Table 3.1, each training transaction i with true label $y_i \in \{0, 1\}$ is assigned an individual sample weight w_i:

$$w_i = \begin{cases} a_i & \text{if } y_i = 1 \text{ (fraudulent transaction, weighted by amount } a_i), \\ C_a & \text{if } y_i = 0 \text{ (legitimate transaction, weighted by review cost } C_a), \end{cases} \tag{6.1}$$

where $y_i = 1$ denotes actual fraud, $y_i = 0$ denotes an actual legitimate transaction, and $C_a = 2.50\,\text{EUR}$ serves as the baseline administrative review cost.

Because loss reweighting alters the empirical class distribution, predicted probability outputs become systematically shifted toward higher values [9], [10]. Consequently, all probabilities from the class-weighted and amount-weighted variants were recalibrated via isotonic regression fitted strictly on the validation partition prior to applying decision rules [9].

6.1.2 Evaluated Decision Rules

Final classifications on the held-out test sets are assigned according to three decision mechanisms:

1. **Default threshold ($t = 0.5$):** The standard symmetric decision boundary.
2. **Global cost-optimal threshold (t^*):** A single global decision threshold selected via a 1,001-point grid search over $[0, 1]$ that directly minimises total financial loss on the *validation partition* (Listing 6.1), applied unchanged to the test set:

$$t^* = \arg\min_{t \in [0,1]} \text{Cost}\left(y_{\text{val}}, \mathbb{I}(\hat{p}_{\text{val}} \geq t), a_{\text{val}}, C_a\right). \tag{6.2}$$

   Unlike analytic rules, t^* adapts empirically to the classifier's actual output distribution.

3. **Example-dependent Bayes threshold ($p^*(x) = C_a/a$):** The transaction-specific minimum-risk rule derived in Equation 3.3. Transaction i is declined if and only if $\hat{p}_i \geq C_a/a_i$. The threshold is inversely proportional to the amount at risk: high-value transactions are declined under weak suspicion, whereas low-value transactions require near certainty.

```
1  def cost_optimal_threshold(y_true, proba, amount, Ca, grid=1001):
2  #Global threshold minimising total cost, selected on the VALIDATION set.
3      ts = np.linspace(0.0, 1.0, grid)
4      best_t, best_c = 0.5, np.inf
5      for t in ts:
6          y_pred = (proba >= t).astype(int)
7          c = total_cost(y_true, y_pred, amount, Ca)
8          if c < best_c:
9              best_c, best_t = c, t
10     return best_t
```

Listing 6.1: Selection of the global cost-optimal threshold t^* on the validation partition.

6.1.3 Financial Evaluation Metrics

The total financial cost on the test partition is computed according to Bahnsen et al. [7]:

$$\text{Cost} = \sum_{i \in \text{FN}} a_i + C_a \cdot (|\text{FP}| + |\text{TP}|), \tag{6.3}$$

where a_i is the individual transaction amount lost on missed frauds (false negatives), and C_a is the administrative overhead incurred for each flagged transaction (false positives and true positives). Correctly approved legitimate transactions (true negatives) incur zero cost.

Model performance is benchmarked against the financial baseline of approving all transactions ($\text{Cost}_{\text{approve-all}} = \sum_i$

6.2 Experimental Results

For each dataset and model, three training variants (baseline, class-weighted, amount-weighted) were evaluated across the three decision rules at $C_a \in \{1.0, 2.5, 5.0\}$ EUR. All thresholds were selected on validation data, and final metrics were computed on the held-out test partitions.

6.2.1 First Attempt: The Savings-Optimal Amount Rule at $C_a = 2.50$

The natural starting point is to apply the example-dependent rule $p^*(x) = C_a/x$ at the primary cost $C_a = 2.50$ EUR and select the configuration of highest *savings*, the quantity the cost model is built to optimise. Table 6.1 reports the result against the unweighted baseline.

Table 6.1: First attempt: baseline versus the highest-savings configuration under the amount-dependent rule ($p^*(x) = C_a/x$, $C_a = 2.50$ EUR). FN = missed frauds, FP = false alarms.

Dataset / Model	Configuration	Recall	Precision	FN	FP	Savings
IEEE / XGBoost	baseline ($t = 0.5$)	0.602	0.922	1647	209	0.522
	savings-optimal	0.869	0.214	542	13 193	**0.870**
IEEE / Neural Net	baseline ($t = 0.5$)	0.336	0.844	2743	257	0.281
	savings-optimal	0.783	0.120	898	23 643	**0.802**
ULB / XGBoost	baseline ($t = 0.5$)	0.747	0.986	24	1	0.730
	savings-optimal	0.484	0.329	49	94	**0.797**
ULB / Neural Net	baseline ($t = 0.5$)	0.716	0.907	27	7	0.571
	savings-optimal	0.484	0.484	49	49	**0.805**

By its own metric this configuration succeeds: savings rise for every model. Yet inspection

of the confusion counts reveals two problems that make it unsuitable as a final result. On IEEE-CIS, the reduction in missed frauds (1647 → 542; 2743 → 898) is bought with an explosion of false alarms (209 → 13 193; 257 → 23 643) and a collapse of precision to 0.12–0.21. On ULB the outcome is worse still: the configuration *increases* missed frauds (24 → 49; 27 → 49) and halves recall, improving savings only by concentrating on a handful of high-value frauds.

The cause is structural. The rule $p^*(x) = C_a/x$ prices a false positive at only $C_a = 2.50$ EUR, so it declines large-amount transactions on very weak evidence (the IEEE precision collapse), while for any amount at or below C_a the threshold reaches 1.0 and the transaction is never declined, so the many low-value frauds in ULB are deliberately forgone. In short, *savings measure avoided monetary loss, not the number of frauds caught*, and at a fixed low $C_a = 2.50$ the two objectives diverge. Since the aim of this work is the reduction of false negatives, the savings-optimal rule alone is inadequate, and two refinements were pursued.

6.2.2 Refinement: The Global Cost-Optimal Threshold

The first refinement replaces the analytic per-transaction rule with the global cost-optimal threshold t^* (Listing 6.1), which is fitted to the model's own probability distribution rather than imposed by the cost matrix. The second sweeps the administrative cost C_a, which, as Section 6.2.3 shows, controls how conservative t^* is. Selecting, for each model, the t^*/C_a combination that reduces false negatives while keeping precision acceptable yields Table 6.2.

Table 6.2: Refined result: baseline versus the best false-negative-reducing configuration using the global threshold t^* (with the C_a at which it is obtained). XGB*=XGBoost, NN*=Neural Network

Dataset/Model	Configuration	Recall	Precision	FN	FP	Savings
IEEE/XGB*	baseline ($t = 0.5$)	0.602	0.922	1647	209	0.522
	baseline + t^* (C_a=5)	0.872	0.303	528	8 281	0.764
IEEE/NN*	baseline ($t = 0.5$)	0.336	0.844	2743	257	0.281
	class-weight + t^* (C_a=5)	0.799	0.151	829	18 538	0.627
ULB/XGB*	baseline ($t = 0.5$)	0.747	0.986	24	1	0.730
	baseline + t^* (C_a=5)	**0.779**	0.925	**21**	6	0.716
ULB/NN*	baseline ($t = 0.5$)	0.716	0.907	27	7	0.571
	class-weight + t^* (C_a=1)	**0.779**	0.851	**21**	13	0.737

The picture changes markedly, and on ULB it reverses. For both ULB models the global threshold now *reduces* missed frauds below the baseline (24 → 21 and 27 → 21) while retaining high precision (0.93 and 0.85) and only a handful of false alarms (6 and 13). The regression seen in Section 6.2.1 was thus an artefact of the amount-dependent rule, not of

cost-sensitivity as such: once the decision rule is fitted to the data rather than to the raw cost matrix, cost-sensitive learning does reduce false negatives on ULB.

On IEEE-CIS the refinement does not remove the trade-off but makes it explicit and controllable. The best configuration cuts missed frauds by roughly two thirds (1647 $\rightarrow$ 528 for XGBoost) at a precision of 0.30 , far better than the 0.21 of the savings-optimal rule for a comparable recall. A genuine tension remains: on this dataset, catching substantially more fraud necessarily raises the false-alarm count, because much of the fraudulent value sits in transactions the model cannot separate cleanly. The extent of the reduction is therefore a policy choice, quantified by the sensitivity analysis below.

6.2.3 Sensitivity to the Administrative Cost C_a

Table 6.3 reports the best achievable savings as C_a varies. Beyond savings, C_a is the knob governing the precision–recall balance of the threshold: a larger C_a makes the cost-optimal policy more conservative, raising precision and cutting false alarms, while a smaller C_a trades precision for recall. This is visible for ULB/XGBoost with the global threshold: at $C_a = 1.0$ it reaches recall 0.85 (only 14 missed frauds) but precision 0.39; at $C_a = 5.0$ it settles at recall 0.78, precision 0.93 and 6 false alarms. The model ordering is stable across the range, which supports the robustness of the comparison: the conclusions do not depend on the precise, business-specific value of C_a.

Table 6.3: Best achievable savings as a function of the administrative cost C_a (EUR).

Dataset / Model	$C_a = 1.0$	$C_a = 2.5$	$C_a = 5.0$
IEEE / XGBoost	0.922	0.870	0.808
IEEE / Neural Net	0.891	0.802	0.711
ULB / XGBoost	0.806	0.797	0.789
ULB / Neural Net	0.812	0.805	0.799

6.2.4 Two Objectives: Minimising Loss vs. Minimising Missed Frauds

The progression above isolates the central lesson of the chapter. Cost-sensitive learning optimises the objective it is given. Optimising *savings* with the analytic amount rule maximises avoided euros and, at a low C_a, is willing both to flood the review queue with false alarms and to ignore low-value frauds, which is why it raised false negatives on ULB. Optimising with the *global cost-optimal threshold*, tuned on the data, instead reduces the *count* of missed frauds, which is the stated goal of this work. Neither is "correct" in the abstract: the appropriate operating point depends on whether the institution seeks to minimise monetary loss or the number of undetected frauds, and on its tolerance for false alarms. The value of the cost-sensitive framework is that it makes this choice explicit and

lets a single trained model be moved along the trade-off by the cost parameter, without retraining.

6.3 Comparison: XGBoost vs. Neural Network

XGBoost dominates the neural network at every stage. On its cost-insensitive baseline it already achieves higher savings and recall (IEEE-CIS: 0.522 vs. 0.281 savings, 0.602 vs. 0.336 recall), consistent with the broader finding that gradient-boosted trees outperform neural networks on tabular data [11]. Under cost-sensitivity both improve, but XGBoost does so at a much lower precision cost: for a comparable recall on IEEE-CIS it produces roughly 8 000 false alarms against the network's 18 000–24 000. The network shows the larger *relative* gain (its weak baseline leaves more room), so in the terms of the research question its decisions are more sensitive to cost-oriented adaptation; but it remains the less precise and overall weaker model, and on IEEE-CIS it never reaches a usable precision at high recall.

6.4 Comparison: IEEE-CIS vs. ULB

The two datasets lead to different, and complementary, conclusions. On ULB, cost-sensitivity with the global threshold reduces missed frauds below the baseline while keeping precision above 0.85, a clean improvement. On IEEE-CIS it reduces missed frauds by up to two thirds, but only by accepting a large rise in false alarms; the objectives of catching fraud and sparing legitimate customers are in genuine tension there. The determining factor is not the imbalance ratio, ULB is the more imbalanced, but the amount distribution of fraud relative to C_a and how separable the high-value frauds are. A practical implication is that the cost-sensitive operating point should be chosen per portfolio: on data resembling ULB, the global cost-optimal threshold delivers both fewer missed frauds and high precision, whereas on data resembling IEEE-CIS the institution must decide explicitly how many false alarms it will tolerate for each additional fraud caught.

7 Discussion

This chapter interprets the results of Chapters 5 and 6 in light of the research question: whether cost-sensitive learning can reduce false negatives in fraud detection while keeping the number of false positives low, and how XGBoost and a neural network compare in doing so. It draws the findings together, explains why the two datasets behaved differently, states the limitations of the study honestly, and outlines directions for further work.

7.1 Summary of Findings

The central finding is that cost-sensitive learning can reduce false negatives, but that whether it does so cleanly or only through a precision–recall trade-off is governed by the decision rule and by the characteristics of the data, not by cost-sensitivity in the abstract.

Three results support this. First, the cost-insensitive baselines confirmed that a default 0.5 threshold leaves a large share of fraud undetected: XGBoost missed 1,647 of 4,133 frauds on IEEE-CIS and the neural network missed 2,743, while on ULB the two models missed 24 and 27 of 95 frauds respectively. High precision at the default threshold is therefore bought at the cost of recall, which is precisely the behaviour the cost-sensitive framework of Chapter 3 is designed to correct.

Second, the analytic amount-dependent rule $p^*(x) = C_a/x$, although it maximised monetary savings, proved an unreliable route to the stated goal. On IEEE-CIS it reduced missed frauds only by raising false alarms into the tens of thousands and collapsing precision to between 0.12 and 0.21; on ULB it *increased* missed frauds from 24 to 49 (XGBoost) and 27 to 49 (neural network). This exposed a distinction that is easy to overlook: savings measure avoided monetary loss, not the number of frauds caught, and at a low fixed C_a the two objectives diverge.

Third, replacing the analytic rule with a global cost-optimal threshold t^*, fitted to each model's own probability distribution on the validation set, reconciled cost-sensitivity with the reduction of false negatives. On ULB both models then reduced missed frauds below their baselines ($24 \rightarrow 21$ and $27 \rightarrow 21$) while retaining precision of 0.93 and 0.85. On IEEE-CIS the same rule cut missed frauds by roughly two thirds ($1{,}647 \rightarrow 528$ for XGBoost) but only at a precision of 0.30, making the trade-off explicit and controllable rather than removing it.

7.2 Interpretation

Why the amount rule backfired. The rule $p^*(x) = C_a/x$ prices a false positive at only the fixed administrative cost, so it declines high-value transactions on very weak evidence and, for any transaction whose amount does not exceed C_a, sets a threshold of one and never declines it. This is the correct Bayes-minimum-risk decision for the cost matrix as written [7], [10], but the objective it optimises, avoided loss, is not the same as the count of frauds caught. The global threshold t^* succeeds because it is chosen empirically against the data rather than imposed by the raw cost matrix, which is consistent with Elkan's recommendation that, for learners whose probability estimates are imperfectly calibrated, the operating threshold be found empirically rather than trusted from the nominal costs [10].

Why XGBoost outperformed the neural network. XGBoost was the stronger model at every stage. It achieved higher baseline recall than the neural network, most markedly on IEEE-CIS (0.602 versus 0.336), and under cost-sensitivity reached comparable recall at far lower false-alarm cost (roughly 8,000 false positives against the network's 18,000 to 24,000 for a similar recall on IEEE-CIS). This is consistent with the broader evidence that gradient-boosted trees remain strong on tabular data [11]. A likely contributor specific to IEEE-CIS is its extensive missingness and high-cardinality categorical features, which XGBoost handles natively through sparsity-aware split finding [6] and its tolerance of incomplete data [5], whereas the neural network's imputation- and encoding-based pipeline must reconstruct this information. The network did show the larger *relative* improvement under cost-sensitivity, because its weaker baseline left more room, but it never became the more precise or overall stronger model.

Why the two datasets diverged. The most instructive contrast is between the datasets. On ULB, cost-sensitivity was a clean improvement; on IEEE-CIS it was a genuine trade-off. The determining factor was not the class-imbalance ratio, ULB is in fact the more imbalanced, at 0.17% fraud against 3.5%, but the separability of fraud in the feature space. ULB's features are clean, complete, and already reduced to informative principal components, so its fraud is more separable and can be caught without flooding the review queue; the baseline PR-AUC values reflect this, being higher and far more stable across models on ULB (0.832 and 0.810) than on IEEE-CIS (0.797 and 0.552) [18]. On IEEE-CIS much of the fraudulent value sits in transactions the model cannot separate cleanly, so catching more of it necessarily raises false alarms. The practical reading is that the cost-optimal operating point should be selected per portfolio rather than assumed to transfer across datasets.

7.3 Limitations

Several limitations qualify these findings and should be kept in view.

First, the IEEE-CIS feature set is largely anonymised: the meanings of the `V`, `C`, and `D` feature blocks are not published. This precludes domain-driven feature engineering and therefore caps how separable fraud can be made on that dataset. The ceiling observed on IEEE-CIS is thus in part a property of the data rather than of the methods, and improvements were necessarily limited to generic, model-side techniques.

Second, missing numeric values were handled by single median imputation. This was a deliberate, scalable choice given the size of the data, but single imputation does not propagate the additional uncertainty introduced by the missing values and can understate the variance of the affected features [5]. The engineered missingness indicators mitigate this only for the fact of a value being absent, not for the imputed magnitude.

Third, the neural network was held at a single fixed architecture rather than tuned by a search comparable to the one applied to XGBoost. This was intentional, so that differences between the models could be attributed to the cost-sensitive mechanism rather than to unequal tuning effort; but it means the network's results represent a reasonable fixed configuration rather than its best attainable performance, and its weaker showing should be read with that in mind.

Fourth, the administrative cost C_a is a business-specific quantity that is not publicly disclosed. It was set to a plausible primary value and varied over a range as a sensitivity analysis, and the model ordering was stable across that range, but the absolute savings figures depend on this assumption and would shift under a materially different cost.

Finally, the study rests on two datasets. Although they were chosen for their complementary characteristics, two datasets cannot establish that the conclusions generalise to the full diversity of real transaction streams, which also exhibit concept drift over time [9] that was outside the scope of this static evaluation.

8 Conclusion

This thesis asked whether cost-sensitive learning can reduce false negatives in financial fraud detection, and how XGBoost and a neural network compare in doing so. Working from two publicly available datasets of contrasting character, the large, partially interpretable but heavily missing IEEE-CIS dataset and the smaller, clean, PCA-anonymised ULB dataset, it built a leakage-aware pipeline, established cost-insensitive baselines, and evaluated a cost-sensitive framework based on the expected-loss theory of Elkan and the amount-based cost matrix of Bahnsen et al.

The answer to the research question is a qualified yes. Cost-sensitive learning can reduce the number of missed frauds relative to a cost-insensitive baseline, but only when the decision rule is fitted to the data rather than imposed analytically. The purely savings-optimal amount rule maximised avoided monetary loss yet failed the stated objective, on ULB even increasing missed frauds, because maximising savings is not the same as minimising the count of undetected frauds. Replacing it with a global cost-optimal threshold selected on a validation set reconciled the two: on ULB it reduced missed frauds below the baseline for both models while keeping precision high, and on IEEE-CIS it cut missed frauds by roughly two thirds, though there only by accepting more false alarms.

On the model comparison, XGBoost was the stronger classifier throughout. It achieved higher recall than the neural network on both datasets and, under cost-sensitivity, reached comparable recall at a substantially lower false-alarm cost. This is consistent with the wider evidence that gradient-boosted trees remain strong on tabular data [11], and it was most pronounced on the messier IEEE-CIS dataset, whose missingness and categorical features XGBoost handles natively.

The main contribution of this work is to show that the outcome of cost-sensitive fraud detection is governed less by the class-imbalance ratio than by the separability of fraud in the feature space and by the choice of decision rule. On clean, separable data, cost-sensitivity delivered fewer missed frauds and high precision at once; on noisier data it exposed a genuine precision–recall trade-off that the cost parameter makes explicit and controllable. The practical consequence is that a single trained model can be moved along this trade-off by its cost parameter without retraining, and that the operating point should be chosen for the specific portfolio rather than assumed to transfer.

Several directions would extend these findings. Broadening the study to further datasets and to deep architectures designed for tabular data would test whether the neural net-

work's disadvantage narrows on inputs better suited to it. Enriching the cost model beyond the transaction amount, for example with recovery probabilities or downstream operational costs, would let the cost matrix reflect the true economics more faithfully. More sophisticated handling of missing values, such as multiple imputation, could be weighed against the lightweight single imputation used here. Finally, a temporal evaluation that accounts for concept drift [9] would test whether the cost-optimal threshold remains stable as fraud patterns evolve, which is essential for any deployment beyond a static benchmark.

Bibliography

[1] A. Abdallah, M. A. Maarof, and A. Zainal, "Fraud detection system: A survey," *Journal of Network and Computer Applications*, vol. 68, pp. 90–113, 2016. DOI: 10.1016/j.jnca.2016.04.007.

[2] F. K. Alarfaj, I. Malik, H. U. Khan, N. Almusallam, M. Ramzan, and M. Ahmed, "Credit Card Fraud Detection Using State-of-the-Art Machine Learning and Deep Learning Algorithms," *IEEE Access*, vol. 10, pp. 39 700–39 715, 2022. DOI: 10.1109/ACCESS.2022.3166891.

[3] B. Baesens, S. Höppner, and T. Verdonck, "Data engineering for fraud detection," *Decision Support Systems*, vol. 150, p. 113 492, 2021. DOI: 10.1016/j.dss.2021.113492.

[4] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012. [Online]. Available: https://www.jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf.

[5] H. Cevallos Valdiviezo and S. Van Aelst, "Tree-based prediction on incomplete data using imputation or surrogate decisions," *Information Sciences*, vol. 311, pp. 163–181, 2015. DOI: 10.1016/j.ins.2015.03.018.

[6] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. [Online]. Available: https://arxiv.org/abs/1603.02754.

[7] A. Correa Bahnsen, A. Stojanovic, D. Aouada, and B. Ottersten, "Cost Sensitive Credit Card Fraud Detection using Bayes Minimum Risk," in *2013 12th International Conference on Machine Learning and Applications (ICMLA)*, vol. 1, IEEE, 2013, pp. 333–338. DOI: 10.1109/ICMLA.2013.68.

[8] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3784–3797, 2018. DOI: 10.1109/TNNLS.2017.2736643.

[9] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating Probability with Undersampling for Unbalanced Classification," in *2015 IEEE Symposium Series on Computational Intelligence (SSCI)*, IEEE, 2015, pp. 159–166. DOI: 10.1109/SSCI.2015.33.

[10] C. Elkan, "The Foundations of Cost-Sensitive Learning," in *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI-01)*, San Diego, CA, 2001, pp. 973–978. DOI: 10.5555/1642194.1642301.

[11] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why Do Tree-Based Models Still Outperform Deep Learning on Tabular Data?" In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. DOI: 10.48550/arXiv.2207.08815.

[12] O. Lyashevska, F. Malone, E. MacCarthy, J. Fiehler, J.-H. Buhk, and L. Morris, "Class Imbalance in Gradient Boosting Classification Algorithms: Application to Experimental Stroke Data," *Statistical Methods in Medical Research*, vol. 30, no. 3, pp. 916–925, 2021. DOI: 10.1177/0962280220980484.

[13] Machine Learning Group – Université Libre de Bruxelles (MLG-ULB). Credit Card Fraud Detection Dataset, Kaggle, Accessed: Sep. 2, 2026. [Online]. Available: https://www.kaggle.com/mlg-ulb/creditcardfraud.

[14] P. Madley-Dowd, R. Hughes, K. Tilling, and J. Heron, "The proportion of missing data should not be used to guide decisions on multiple imputation," *Journal of Clinical Epidemiology*, vol. 110, pp. 63–73, 2019. DOI: 10.1016/j.jclinepi.2019.02.016.

[15] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://www.jmlr.org/papers/v12/pedregosa11a.html.

[16] C. Phua, V. Lee, K. Smith, and R. Gayler, "A comprehensive survey of data mining-based fraud detection research," *arXiv preprint arXiv:1009.6119*, 2010. Accessed: Sep. 6, 2026. [Online]. Available: https://arxiv.org/abs/1009.6119.

[17] Y. Sahin, S. Bulkan, and E. Duman, "A cost-sensitive decision tree approach for fraud detection," *Expert Systems with Applications*, vol. 40, no. 15, pp. 5916–5923, 2013. DOI: 10.1016/j.eswa.2013.05.021.

[18] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. DOI: 10.1371/journal.pone.0118432.

[19] Statista. Card Fraud Worldwide and in the U.S.: Value of Losses 2014–2023, with Forecasts to 2028. Original data source: The Nilson Report. Chapter 09, Slide 52, Statista Dossier, *Credit Cards in the United States*, Accessed: Sep. 2, 2026. [Online]. Available: https://www.statista.com/study/12118/credit-cards-in-the-united-states-statista-dossier/.

[20] Vesta Corporation and IEEE Computational Intelligence Society. IEEE-CIS Fraud Detection Dataset, Kaggle, Accessed: Sep. 2, 2026. [Online]. Available: https://www.kaggle.com/c/ieee-fraud-detection.

Software and Tools Used

To ensure complete academic transparency, the following software platforms, computational environments, programming libraries, datasets, and AI-assisted tools were used during the research and writing of this thesis:

AI and Translation Assistants

- **ChatGPT, Claude, and Gemini:** Used as auxiliary assistants for conceptual brainstorming, structuring arguments, refining academic prose and phrasing, and clarifying complex technical implementations. All model design, experiments, results, and conclusions are the author's own.
- **DeepL:** Used for translation support and phrasing refinement.

Platforms and Hardware Environments

- **Kaggle:** Used for data hosting and as the execution environment, providing the NVIDIA Tesla P100 GPU on which the models were trained.
- **Jupyter Notebook:** Interactive computing environment in which the experiments were developed and run.
- **Overleaf (LaTeX):** Used for document authoring, typesetting, and bibliography management.
- **Git / GitHub:** Used for version control and backup of code and thesis files.
- **DataCamp:** Used as a self-study resource for machine learning and Python.

Programming Language and Core Libraries

- **Python 3.12:** Core programming language used for the entire experimental pipeline.
- **pandas & NumPy:** Used for tabular data manipulation, merging, feature engineering, missingness analysis, and vectorized numerical operations.
- **scikit-learn:** Used for the stratified train/validation/test split, standardization (`StandardScaler`), randomized hyperparameter search (`RandomizedSearchCV`) with

stratified cross-validation, evaluation metrics (precision, recall, average precision, confusion matrix), and probability calibration (`IsotonicRegression`).

- **XGBoost:** Used to implement the gradient-boosting baseline and the cost-sensitive classification models, including native handling of categorical features.
- **TensorFlow / Keras:** Used for building, training, and evaluating the feedforward neural network models.
- **Matplotlib:** Used for the exploratory data-analysis figures and the confusion-matrix visualizations.

Datasets

- **IEEE-CIS Fraud Detection Dataset:** `https://www.kaggle.com/competitions/ieee-fraud-detection/data`
- **Credit Card Fraud Detection Dataset (ULB):** `https://www.kaggle.com/mlg-ulb/creditcardfraud`